

**DOCUMENT MANUAL APLIKASI  
ZAFITECH**

**Desain Arsitektur, Struktur Folder, Implementasi NestJs, PostgreSQL dan Prisma, RBAC Enterprise-Ready untuk Asisten AI Zafi, hingga setup GitHub dan CI/CD.**

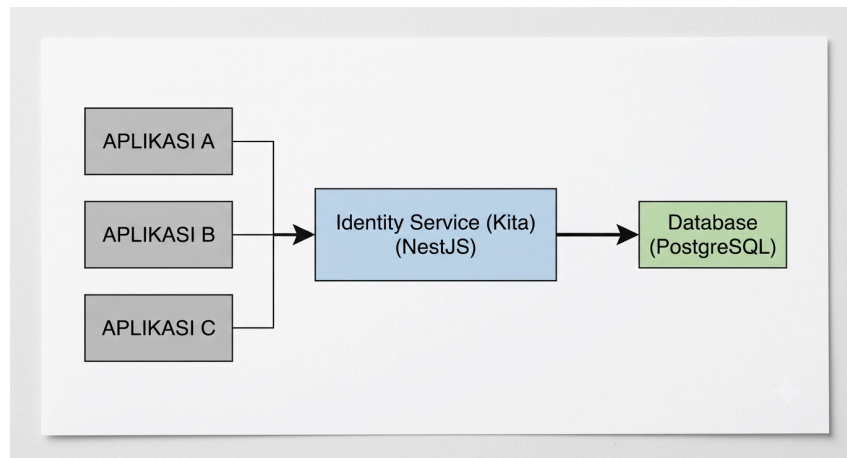
## A.KONSEP DASAR DAN TUJUAN

Aplikasi ini dibangun menggunakan pendekatan **identity service** seperti sistem digital terpusat, contoh :

1. Google Identity = Satu akun untuk semua layanan Google
2. Login dengan Google/Facebook = Satu identitas untuk banyak aplikasi

Kita membuat sistem yang sama, tapi untuk kebutuhan internal perusahaan/organisasi.

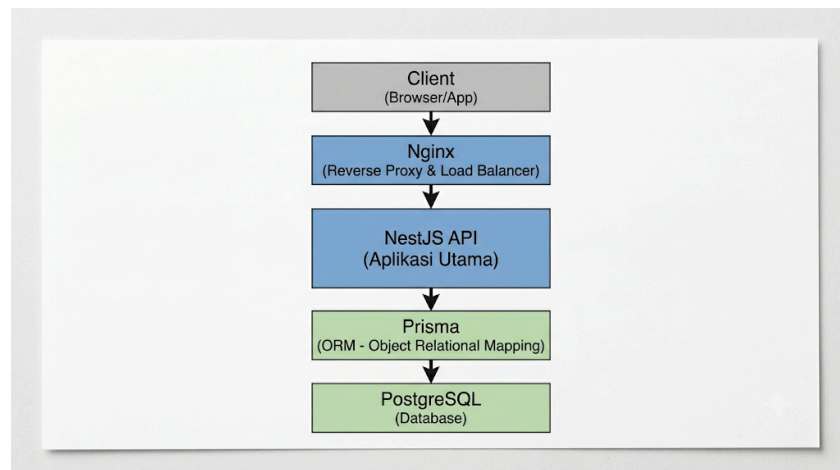
Bentuk identity service yang dibangun contoh arsitekturnya seperti ini:



Gambar 1. Bentuk Identity Service

Semua aplikasi menggunakan satu pusat autentikasi dan otorisasi.

## B.ARSITEKTUR PRODUKSI



Gambar 2. Arsitektur Produksi

Penjelasan Layer:

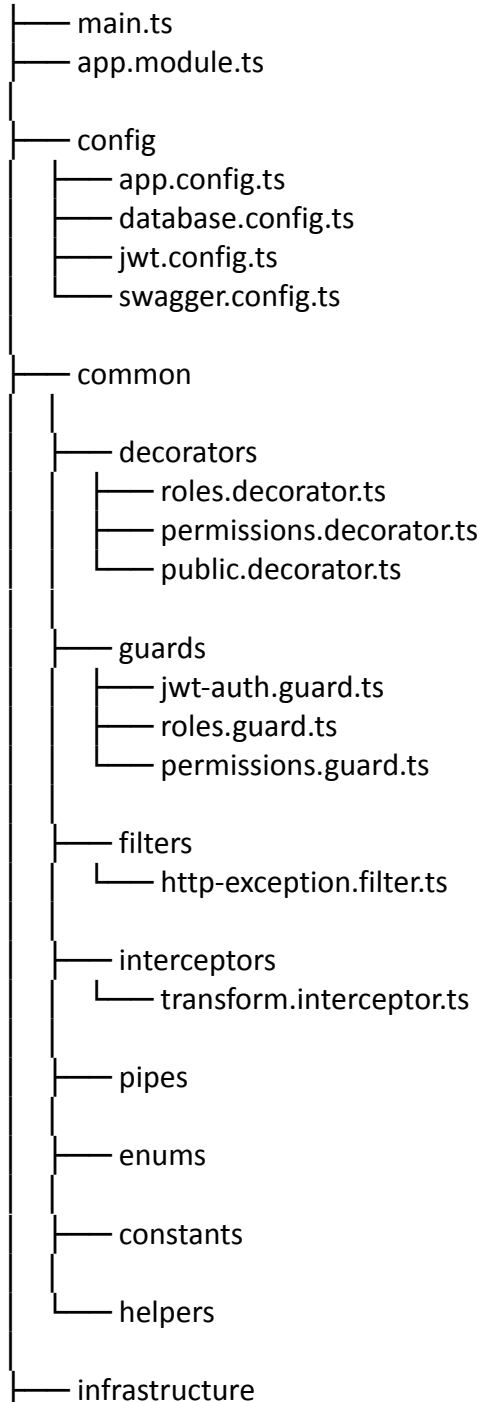
1. Client - Aplikasi yang mengakses (Web, Mobile, Desktop)
2. Nginx - Seperti resepsionis hotel: menerima tamu, mengarahkan ke ruangan yang tepat
3. NestJS API - Restoran: tempat semua logika bisnis terjadi
4. Prisma - Koki yang menghubungkan restoran dengan gudang bahan (database)
5. PostgreSQL - Gudang penyimpanan data

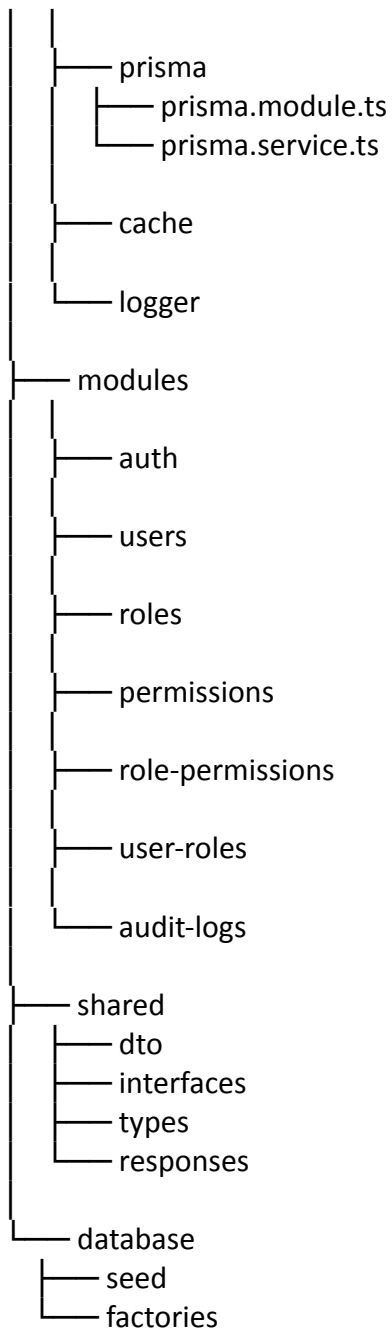
## Target Arsitektur

### C.STRUKTUR FOLDER DETAIL

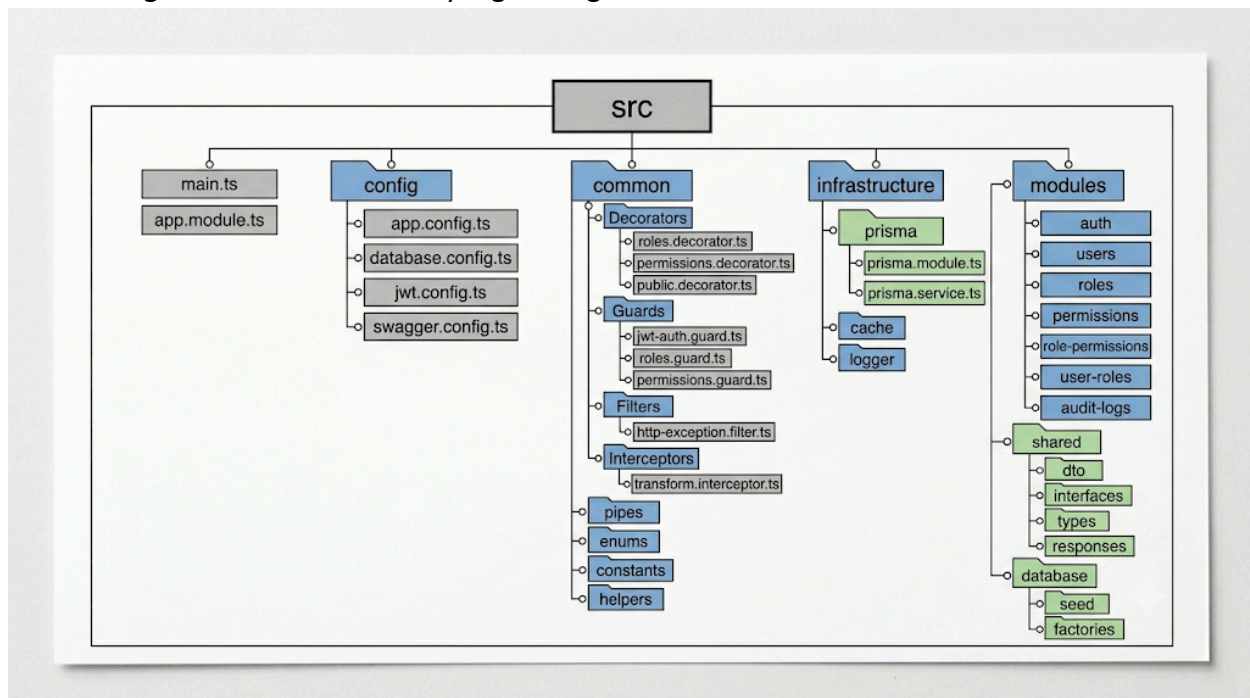
Project ini menggunakan enterprise kecil dan menengah. Dimana struktur folder yang dibangun pada tahapan ini dalam bentuk folder seperti ini :

src





Berikut ini gambar struktur folder yang dibangun :



Gambar 3. Struktur Folder Identity Service

Pendekatan ini berhasil menggabungkan fleksibilitas Feature-Based (Modular) dengan kerapian Clean Architecture .Berikut adalah penjelasan detail fungsi dari masing-masing folder dalam struktur final :

### 1.Root Folder (src/)

Ini adalah pusat dari seluruh aplikasi.

- [main.ts](#)
- [app.module.ts](#)

**main.ts:** Entry point utama aplikasi. Tempat untuk bootstrap NestJS, mengatur port server, mengaktifkan global middleware (seperti ValidationPipe, CORS), serta menginisialisasi Swagger.

**app.module.ts:** Modul pusat (root) aplikasi. Semua modul fitur (modules/\*), konfigurasi (config/\*), dan infrastruktur (infrastructure/\*) akan diimpor dan disatukan di sini.

### 2.config/ (Centralized Configuration)

Digunakan untuk mengisolasi pembacaan environment variables (.env). Alih-alih memanggil process.env secara acak di seluruh aplikasi, konfigurasi dibaca terpusat menggunakan @nestjs/config. **Fungsi:** Menyimpan semua pengaturan aplikasi. Seperti apa? Panel kontrol di pesawat

- [app.config.ts](#)
- [database.config.ts](#)
- [jwt.config.ts](#)
- [swagger.config.ts](#)

**app.config.ts:** Menyimpan konfigurasi dasar aplikasi (seperti nama aplikasi, port, environment development/production, atau versi API).

**database.config.ts:** Menyimpan kredensial database PostgreSQL untuk kebutuhan Prisma atau koneksi eksternal.

**jwt.config.ts:** Menyimpan rahasia (secret key) JWT dan durasi kedaluwarsa token untuk modul Autentikasi.

**swagger.config.ts:** Menyimpan konfigurasi tampilan OpenID/Swagger untuk dokumentasi API.

### **3.common/ (Cross-Cutting Concerns & Framework Specifics)**

Berisi aset-aset global yang spesifik dengan fitur bawaan NestJS (request-response lifecycle) dan dipakai secara berulang di berbagai modul.

**Fungsi:** Kode yang digunakan di banyak tempat

**Analog:** Seperti peralatan umum di kantor (fotokopi, printer)

- decorators/
- guards/
- filters/
- interceptors/
- pipes/
- enums/
- constants/
- helpers/

**decorators/:** Metadata penKita. Contohnya @Public() untuk membypass JWT, serta @Roles() dan @Permissions() untuk mengunci endpoint berdasarkan hak akses.

**guards/:** Gerbang keamanan. jwt-auth.guard.ts memeriksa validitas token, sementara roles.guard.ts dan permissions.guard.ts memeriksa apakah user memiliki hak akses yang sesuai untuk masuk ke controller.

**filters/:** Penanganan Error atau Penyeragaman error. http-exception.filter.ts menangkap semua error aplikasi (seperti 404 atau 500) lalu mengubah format responnya agar seragam di mata klien.

**interceptors/:** Manipulasi data sebelum/sesudah masuk controller. transform.interceptor.ts biasanya digunakan untuk membungkus respon sukses menjadi format stKitar (contoh: { success: true, data: [...] }).

**pipes/:** Untuk validasi input data tingkat lanjut atau transformasi tipe data otomatis (misal: mengubah string ID menjadi objek).

**enums/:** Tempat menyimpan data enumerasi global (contoh: enum RoleType { ADMIN = 'ADMIN', USER = 'USER' }).

**constants/:** Menyimpan string/nilai statis yang tidak pernah berubah (seperti nama antrean, nama strategy, atau pesan error stKitar).

**helpers/:** Fungsi-fungsi utilitas murni tanpa efek samping (contoh: fungsi kalkulasi tanggal, pemformat teks, dll).

#### 4.infrastructure/ (Low-Level Drivers & Tools)

Lapisan luar yang berhubungan langsung dengan pustaka eksternal, framework, atau sistem operasi (I/O operations). **Fungsi:** Komponen teknis yang mendukung aplikasi Analog: Seperti instalasi listrik dan pipa air di gedung

- prisma/
- cache/
- logger/

**prisma/:** Modul pembungkus database. prisma.service.ts mengontrol siklus hidup koneksi ke PostgreSQL, sedangkan prisma.module.ts mengeksport servis ini agar bisa dipakai di tempat lain.

**cache/:** Implementasi manajemen memori sementara (misalnya menggunakan Redis atau in-memory cache) untuk mempercepat performa API.

**logger/:** Kustomisasi sistem pencatatan log aplikasi (seperti integrasi dengan Winston atau Pino) untuk memantau aktivitas server.

#### 5.modules/ (Feature/Domain Layer)

Inti bisnis aplikasi kita. Setiap folder di sini merepresentasikan satu domain/fitur terisolasi yang di dalamnya idealnya menerapkan struktur clean controller-service sendiri. **Fungsi:** Tempat semua fitur utama aplikasi. Analog: Seperti departemen di perusahaan

- auth/
- users/
- roles/
- permissions/
- role-permissions/
- user-roles/
- audit-logs/

**auth/:** Menangani proses login, register, logout, penyerahan token JWT, hingga refresh token.

**users/:** Manajemen data user (CRUD, profil, ganti password).

**roles/:** Manajemen pembuatan role (seperti ADMIN, MANAGER, STAFF).

**permissions/:** Manajemen definisi hak akses granular (seperti create:user, delete:invoice).

**role-permissions/:** Menangani pemetaan (assign) hak akses apa saja yang dimiliki oleh role tertentu.

**user-roles/:** Menangani pemetaan user memiliki role apa saja (mendukung multi-role).

**audit-logs/:** Mencatat rekam jejak aktivitas penting user (siapa melakukan apa dan kapan) untuk kebutuhan audit keamanan.

#### 6.shared/ (Plain Objects & Shared Contracts)

Berisi objek TypeScript murni (plain objects) yang bebas dari dependensi NestJS, digunakan untuk pertukaran tipe data antar lapisan. **Fungsi:** Kode yang digunakan antar modul. Analog: Seperti kantin bersama untuk semua karyawan

- dto/
- interfaces/
- types/
- responses/

**dto/:** Data Transfer Object global (seperti format untuk pagination query { page: 1, limit: 10 }).

**interfaces/**: Kontrak struktur objek global.

**types/**: Gabungan tipe data kustom atau tipe utilitas TypeScript.

**responses/**: Definisi interface bentuk respon API agar seragam di sisi internal pengembang.

## 7.database/ (Data Simulation & Provisioning)

Berada di luar **src/** (atau sejajar di **root**), digunakan untuk kebutuhan manajemen data awal database. **Fungsi**: Script untuk mengisi data awal. **Analog**: Seperti dapur yang menyiapkan makanan

- seed/
- factories/

**seed/**: Skrip untuk mengisi database dengan data awal yang wajib ada saat aplikasi pertama kali dijalankan (seperti membuat akun superadmin pertama dan daftar permissions).

**factories/**: Generator data palsu (menggunakan library seperti Faker) untuk membuat ratusan data tiruan demi kebutuhan testing atau simulasi performa.

Dengan memisahkan infrastructure (**Prisma**) dan **common** (Guards/Filters) dari **modules** (Bisnis Logik), aplikasi kita menjadi sangat kokoh. Jika suatu hari nanti kita ingin mengganti **Prisma** dengan **TypeORM**, atau mengganti **PostgreSQL** dengan **MongoDB**, Kita cukup mengubah isi folder **infrastructure/** tanpa perlu membongkar seluruh kode di dalam folder **modules/**

## D.DATABASE SCHEMA & PRISMA

Skema Prisma ini adalah blueprint database RBAC (**Role-Based Access Control**) yang digunakan untuk siap level production. Skema ini didesain agar satu user bisa memiliki banyak role (multi-role) dan setiap role memiliki banyak hak akses spesifik (granular permissions). Selain itu, skema ini sudah dilengkapi dengan sistem keamanan token dan pelacakan aktivitas (audit trail).

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  name        String
  password    String
  isActive    Boolean     @default(true)
  lastLogin   DateTime?
  createdAt   DateTime     @default(now())
```

```
updatedAt    DateTime    @updatedAt
deletedAt    DateTime?
```

```
userRoles    UserRole[]
refreshTokens RefreshToken[]
auditLogs     AuditLog[]
```

```
@@index([email])
@@index([deletedAt])
}
```

```
model Role {
  id          String      @id @default(cuid())
  name        String      @unique
  description  String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
```

```
userRoles    UserRole[]
rolePermissions RolePermission[]
```

```
@@index([name])
}
```

```
model Permission {
  id          String      @id @default(cuid())
  name        String      @unique
  resource     String
  action       String
  description  String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
```

```
rolePermissions RolePermission[]
```

```
@@unique([resource, action])
@@index([name])
}
```

```
model UserRole {
  id      String  @id @default(cuid())
  userId  String
  roleId  String
  createdAt DateTime @default(now())
```

```

user    User    @relation(fields: [userId], references: [id], onDelete: CASCADE)
role    Role    @relation(fields: [roleId], references: [id], onDelete: CASCADE)

@@unique([userId, roleId])
@@index([userId])
@@index([roleId])
}

model RolePermission {
  id      String  @id @default(cuid())
  roleId  String
  permissionId String
  createdAt DateTime @default(now())

  role    Role    @relation(fields: [roleId], references: [id], onDelete: CASCADE)
  permission Permission @relation(fields: [permissionId], references: [id], onDelete: CASCADE)

  @@unique([roleId, permissionId])
  @@index([roleId])
  @@index([permissionId])
}

model RefreshToken {
  id      String  @id @default(cuid())
  userId  String
  tokenHash String
  expiredAt DateTime
  createdAt DateTime @default(now())

  user    User    @relation(fields: [userId], references: [id], onDelete: CASCADE)

  @@index([userId])
  @@index([expiredAt])
}

model AuditLog {
  id      String  @id @default(cuid())
  userId  String?
  action  String
  module  String
  ipAddress String?
  userAgent String?
  details Json?

```

```

createdAt DateTime @default(now())

user User? @relation(fields: [userId], references: [id])

@@index([userId])
@@index([createdAt])
@@index([module])
}

// Enum untuk role default
enum DefaultRole {
  SUPER_ADMIN
  ADMIN
  MANAGER
  USER
  GUEST
}

```

Berikut adalah penjelasan detail fungsi dari masing-masing model tabel dalam skema :

### **1.model User (Manajemen Pengguna)**

Ini adalah tabel inti untuk menyimpan data akun pengguna aplikasi.

Fitur Menarik:

- id String @id @default(cuid()): Menggunakan CUID (aman dari tebakan hacker dibanding ID angka biasa 1,2,3).
- isActive: Fitur untuk memblokir/mengaktifkan user dengan cepat tanpa menghapus datanya.
- deletedAt: Mendukung Soft Delete. Jadi saat user dihapus, datanya tidak benar-benar hilang dari database, hanya diisi tanggalnya (sangat penting untuk keamanan data enterprise).
- @@index: Mempercepat proses pencarian saat user melakukan login (email) atau saat memfilter data yang belum dihapus (deletedAt).

#### **Attribute User:**

```

id
name
email
password
is_active
last_login
created_at
updated_at
deleted_at

```

## 2.model Role & model UserRole (Manajemen Kelompok Akses)

Tabel ini digunakan untuk mengelompokkan pengguna berdasarkan jabatan atau posisinya.

- **Role:** Menyimpan nama perannya (contoh: "ADMIN", "MANAGER").
- **UserRole:** Ini adalah Tabel **Junction (Penghubung)**. Karena kita menggunakan tabel terpisah, satu User bisa memiliki lebih dari satu Role (misal: Budi adalah MANAGER sekaligus ADMIN).
- Fitur Menarik: **onDelete: CASCADE** memastikan jika data User atau Role dihapus, data penghubung di tabel ini otomatis ikut terhapus agar database tidak "kotor".

### Attribute Role :

id  
name  
description  
created\_at  
Updated\_at

### Attribute UserRole:

user\_id  
role\_id

## 3.model Permission & model RolePermission (Hak Akses Granular)

Di sinilah letak kekuatan sistem keamanan Kita. Daripada mengecek "Apakah dia Admin?", aplikasi Kita akan mengecek "Apakah dia punya izin untuk menghapus user?".

- **Permission:** Menyimpan hak aksi yang sangat spesifik.
  - **resource:** Objeknya (contoh: "USER", "PRODUCT", "INVOICE").
  - **action:** Tindakannya (contoh: "CREATE", "READ", "DELETE").
  - **@@unique([resource, action]):** Mencegah duplikasi hak akses (tidak boleh ada dua aturan USER - CREATE).
- **RolePermission:** Tabel penghubung untuk menentukan Role mana yang berhak mendapatkan Permission apa. Misal: Role ADMIN dihubungkan ke Permission USER - DELETE.

### Attribute Permission:

id  
name  
resource  
Action

### Contoh :

user:create  
user:update  
user:delete  
role:create  
role:update  
role:delete

**Attribute RolePermission :**

role\_id  
permission\_id

**4.model RefreshToken (Keamanan Sesi Login)**

Tabel ini digunakan untuk mendukung sistem autentikasi JWT yang aman (Secure Authentication Flow).

- **Fungsinya:** Ketika Access Token JWT buatan NestJS kedaluwarsa dalam 15 menit, klien/frontend tidak perlu meminta user login ulang. Frontend cukup mengirim RefreshToken yang ada di tabel ini untuk mendapatkan Access Token baru.
- **Fitur Menarik:** tokenHash menyimpan versi enkripsi dari token di database (menghindari kebocoran data jika database diretas) dan expiredAt untuk membatasi masa berlaku sesi login.

**Attribute RefreshToken:**

id  
user\_id  
token\_hash  
expired\_at

**5.model AuditLog (Rekam Jejak Aktivitas / Log Forensik)**

Tabel ini berfungsi seperti kotak hitam pesawat (black box). Setiap tindakan krusial harus dicatat di sini.

- **Fungsinya:** Jika ada data penting yang terhapus secara misterius, Kita bisa melacaknya di sini.
- **Kolom Penting:** action (apa yang dilakukan), module (di folder/fitur mana terjadi), ipAddress & userAgent (melacak lokasi perangkat), dan details Json? (untuk menyimpan data sebelum dan sesudah diubah dalam format JSON).
- **Fitur Menarik:** userId String? bertipe opsional (bisa null). Artinya, tabel ini tetap bisa mencatat aktivitas meskipun dilakukan oleh pengguna yang belum login (contoh: percobaan login ilegal).

**Attribute AudiLog :**

id  
user\_id  
action  
module  
ip\_address  
user\_agent  
created\_at

**6.enum DefaultRole**

Ini adalah daftar konstanta statis di dalam database PostgreSQL. Meskipun belum diikat langsung ke kolom tabel di atas, enum ini sangat berguna sebagai acuan stKitar di sisi NestJS saat Kita melakukan Database Seeding (mengisi data awal sistem).

## 7. Prisma Schema

- User
- Role
- Permission
- UserRole
- RolePermission
- RefreshToken
- AuditLog

Jangan simpan **refresh token plaintext**. Gunakan:  
`bcrypt.hash(refreshToken)`

## 8. USER TABLE

Fungsi: Menyimpan data pengguna

|                         |                                                                    |
|-------------------------|--------------------------------------------------------------------|
| <code>id</code>         | → <code>UUID (Primary Key)</code> - Identitas unik                 |
| <code>email</code>      | → <code>String, Unique</code> - Email login                        |
| <code>name</code>       | → <code>String</code> - Nama pengguna                              |
| <code>password</code>   | → <code>String (Hash)</code> - Password yang di-hash               |
| <code>is_active</code>  | → <code>Boolean</code> - Status aktif/non-aktif                    |
| <code>last_login</code> | → <code>DateTime</code> - Terakhir login                           |
| <code>created_at</code> | → <code>DateTime</code> - Waktu dibuat                             |
| <code>updated_at</code> | → <code>DateTime</code> - Waktu diupdate                           |
| <code>deleted_at</code> | → <code>DateTime</code> - Soft delete ( <code>NULL</code> = aktif) |

## 9. Role Table

Fungsi: Menyimpan peran/level akses

|                          |                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------|
| <code>id</code>          | → <code>UUID, Primary Key</code>                                                                 |
| <code>name</code>        | → <code>String, Unique</code> - Nama role ( <code>SUPER_ADMIN</code> , <code>ADMIN</code> , etc) |
| <code>description</code> | → <code>String?</code> - Deskripsi role                                                          |
| <code>created_at</code>  | → <code>DateTime</code>                                                                          |
| <code>updated_at</code>  | → <code>DateTime</code>                                                                          |

## 10. Permission Table

Fungsi: Menyimpan izin spesifik

```
id          → UUID, Primary Key
name        → String, Unique - "user:create"
resource    → String - "users"
action      → String - "create"
description → String? - Deskripsi
created_at  → DateTime
updated_at  → DateTime
```

### 11. UserRole Table (Many-to-Many)

Fungsi: Menghubungkan User dengan Role

```
id          → UUID, Primary Key
user_id     → UUID - Foreign Key ke User
role_id     → UUID - Foreign Key ke Role
created_at  → DateTime
```

### 12. RolePermission Table (Many-to-Many)

Fungsi: Menghubungkan Role dengan Permission

```
id          → UUID, Primary Key
role_id     → UUID - Foreign Key ke Role
permission_id → UUID - Foreign Key ke Permission
created_at  → DateTime
```

### 13. RefreshToken Table

Fungsi: Menyimpan refresh token

```
id          → UUID, Primary Key
user_id     → UUID - Foreign Key ke User
token_hash  → String - Hash dari refresh token
expired_at  → DateTime - Waktu kadaluarsa
created_at  → DateTime
```

### 14. AuditLog Table

Fungsi: Mencatat semua aktivitas

```

id          → UUID, Primary Key
user_id     → UUID? - Foreign Key ke User (bisa NULL)
action      → String - "CREATE_USER", "DELETE_USER"
module      → String - "USERS", "AUTH"
ip_address  → String? - IP pengguna
user_agent  → String? - Browser/device info
details     → JSON - Data tambahan
created_at  → DateTime

```

### E. ALUR KERJA HAK AKSES SAAT APLIKASI BERJALAN:

Saat seorang User melakukan request ke NestJS API:

1. User masuk membawa token —► Diperiksa oleh RefreshToken.
2. NestJS melihat User ini punya Role apa saja di tabel UserRole.
3. NestJS menarik daftar Permission (bisa melakukan apa saja) yang diikat pada role tersebut di tabel RolePermission.
4. PermissionsGuard di NestJS mencocokkan apakah Permission milik user sesuai dengan dekorator endpoint API.
5. Selesai beraktivitas —► Sistem mencatat jejaknya ke AuditLog

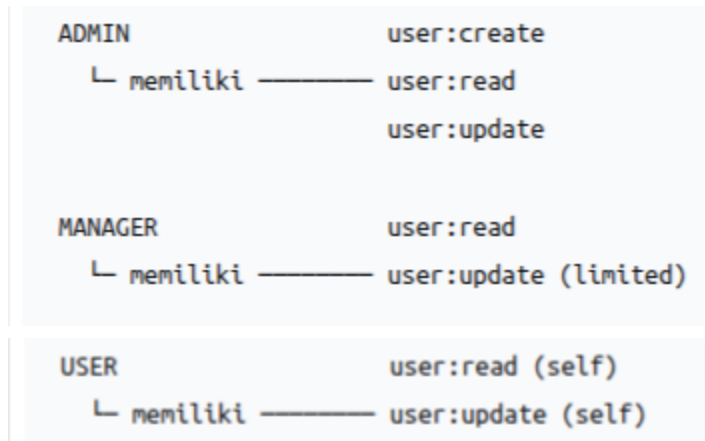
Untuk lebih jelasnya bisa lihat di link ini : [Klik Disini](#).

### F. IMPLEMENTASI RBAC dan PERMISSION

Berikut ini adalah penjelasan mendetail mengenai implementasi sistem otorisasi hibrida RBAC (Role-Based Access Control) yang digabungkan dengan Permissions (Granular Access Control) pada aplikasi NestJS API SERVICE.

Berikut ini konsep Role dan Permission

| ROLE (Peran)   | PERMISSION (Izin) |
|----------------|-------------------|
| SUPER_ADMIN    | user:create       |
| └ memiliki ——— | user:read         |
|                | user:update       |
|                | user:delete       |
|                | role:create       |
|                | role:read         |
|                | role:update       |
|                | role:delete       |



### Bagian 1: Konsep Role vs Permission (Hierarki Akses)

Ini adalah lapisan konseptual atau cetak biru bagaimana hak akses diatur dalam database kita (seperti yang didefinisikan dalam skema Prisma sebelumnya).

- Permissions (Izin): Ini adalah satuan hak akses terkecil dan paling spesifik. Permissions mendefinisikan aksi apa yang bisa dilakukan pada sumber daya tertentu.
  - Contoh: user:create (bisa membuat user), role:delete (bisa menghapus role).
- Role (Peran): Ini adalah kumpulan atau wadah dari beberapa Permissions. Role merepresentasikan jabatan atau posisi user di dalam sistem. Daripada menempelkan 10 permission langsung ke satu user, kita menempelkan 10 permission itu ke satu Role (misal: ADMIN), lalu menempelkan Role ADMIN tersebut ke user.

Tabel tersebut menunjukkan bagaimana Permissions dikelompokkan ke dalam Role:

#### 1.SUPER\_ADMIN:

- Peran: Penguasa tertinggi sistem.
- Izin: Memiliki semua Permissions (user:\*, role:\*). Dia bisa melakukan CRUD (Create, Read, Update, Delete) pada data User maupun data Role.

#### 2.ADMIN

- Peran: Pengelola operasional harian.
- Izin: Sangat kuat, tapi terbatas. Bisa CRUD data User (user:create, user:read, user:update). Namun, di diagram Kita, ADMIN tidak memiliki izin untuk mengelola Role (role:\*) atau menghapus User (user:delete). Ini membatasi ADMIN agar tidak bisa membuat ADMIN baru atau menghapus user lain secara sembarangan.

#### 3.MANAGER

- Peran: Pengawas tim.
- Izin: Terbatas pada domain tertentu. Dia hanya bisa melihat data user (user:read) dan mengubahnya secara terbatas (user:update limited).

#### 4.USER

- Peran: Pengguna biasa.
- Izin: Paling rendah. Dia hanya bisa melihat profilnya sendiri (user:read self) dan mengubah datanya sendiri (user:update self).

## Bagian 2: Implementasi PermissionsGuard (Gerbang Keamanan)

Ini adalah inti logika dari alur kerja hak akses yang sebelumnya kita buat diagramnya. PermissionsGuard bertugas untuk memeriksa apakah request yang masuk diizinkan atau tidak.

### Logika Kode (Langkah demi Langkah):

#### 1. Dependency Injection (constructor):

Guard ini membutuhkan Reflector (untuk membaca dekorator `@Permissions()`), PrismaService (untuk koneksi ke database), dan LoggerService (untuk mencatat log).

#### 2. `canActivate()` (Metode Utama):

Metode ini berjalan setiap kali ada request ke endpoint yang diproteksi. Jika mengembalikan `true`, request dilanjutkan. Jika `false`, request ditolak (403 Forbidden).

#### 3. Langkah 1: Baca Izin yang Diperlukan (Reflector)

TypeScript :

```
const requiredPermissions = this.reflector.get<string[]>(PERMISSIONS_KEY, context.getHandler());
```

Guard membaca izin apa saja yang ditulis di atas metode controller menggunakan dekorator `@Permissions()`.

#### 4. Langkah 2: Cek Jika Endpoint Terbuka

TypeScript :

```
if (!requiredPermissions || requiredPermissions.length === 0) { return true; }
```

Jika endpoint tidak memiliki dekorator `@Permissions()` (atau permission kosong), Guard mengizinkan akses. Ini biasanya digunakan untuk endpoint publik atau endpoint yang hanya butuh login saja ( `JwtAuthGuard`).

#### 5. Langkah 3: Ambil Data User dari Request

TypeScript :

```
const request = context.switchToHttp().getRequest();
const user = request.user;
if (!user) { return false; } // Tolak jika user tidak login
```

Guard mengambil objek user yang sebelumnya telah ditempelkan oleh `JwtAuthGuard` setelah memvalidasi token JWT.

#### 6. Langkah 4: Query Granular ke Database (Tarik Semua Izin User)

TypeScript

```
const userWithPermissions = await this.prisma.user.findUnique({
  where: { id: user.sub }, // ID User dari JWT
  include: {
    userRoles: { // User -> UserRole (Tabel Junction)
```

```

include: {
  role: { // UserRole -> Role
    include: {
      rolePermissions: { // Role -> RolePermission (Tabel Junction)
        include: {
          permission: true, // RolePermission -> Permission
        },
      },
    },
  },
},
};

```

Ini adalah query Prisma yang kompleks untuk menelusuri hierarki: User -> Role-nya -> Permissions-nya. Guard mengambil data user beserta semua objek Permission asli dari database.

### 7.Langkah 5: Kumpulkan Izin ke dalam Set

(dalam kode typescript)

```

const userPermissions = new Set(
  userWithPermissions.userRoles.flatMap(ur =>
    ur.role.rolePermissions.map(rp => rp.permission.name)
  )
);

```

Guard menyederhanakan struktur database yang kompleks tadi menjadi satu datar Set berisi nama-nama permission (contoh: Set { 'user:read', 'user:update' }). Menggunakan Set agar pemeriksaan nanti menjadi sangat cepat dan menangani duplikasi jika user memiliki multi-role.

### 8.Langkah 6: Mencocokkan Izin (Matching Logic)

(dalam kode typescript)

```

const hasAllPermissions = requiredPermissions.every(permission =>
  userPermissions.has(permission));

```

Guard membandingkan: "Apakah semua permission yang diminta di endpoint (every) ada di dalam daftar permission milik user ini (has)?".

### 9.Langkah 7: Audit Log untuk Akses Ditolak

(dalam kode typescript) :

```

if (!hasAllPermissions) { await this.logAccessDenied(...) }

```

Jika izin tidak cocok, Guard memanggil fungsi internal `logAccessDenied` untuk mencatat kejadian ini ke tabel `AuditLog`. Ini penting untuk keamanan sistem.

### 10.logAccessDenied():

Fungsi ini mencatat detail aktivitas ditolak ke database: siapa user-nya (`userId`), aksi (`ACCESS_DENIED`), modul (`AUTHORIZATION`), IP, UserAgent, dan detail JSON berupa path, method, dan permission apa yang dia coba akses.

## Bagian 3: Penggunaan di Controller (Penerapan Hak Akses)

Di sinilah kita menerapkan Gerbang Keamanan tadi di endpoint API kita.

Kode Controller:

### 1.Aktivasi Guard Global di Controller:

TypeScript

```
@UseGuards(JwtAuthGuard, PermissionsGuard)
```

Ini berarti semua endpoint di dalam `UsersController` ini sekarang harus melewati dua lapisan pemeriksaan:

- Lapisan 1 (Authentication): `JwtAuthGuard` memastikan user harus login dan memiliki token JWT yang valid.
- Lapisan 2 (Authorization): `PermissionsGuard` memastikan user yang sudah login tersebut memiliki izin spesifik untuk mengakses endpoint tersebut.

### 2.Mapping Endpoint ke Permission:

- **Post() /users (Create):** Mengunci endpoint pembuat user dengan izin `@Permissions('user:create')`. Berdasarkan hierarki di Part 1, hanya `SUPER_ADMIN` dan `ADMIN` yang bisa memanggil API ini.
- **Get() /users dan Get(':id') (Read):** Mengunci endpoint pembaca data user dengan izin `@Permissions('user:read')`. `SUPER_ADMIN`, `ADMIN`, `MANAGER`, dan `USER` bisa memanggil API ini.
- **Patch(':id') (Update):** Mengunci endpoint pengubah data dengan izin `@Permissions('user:update')`. Berdasarkan hierarki, `SUPER_ADMIN`, `ADMIN`, dan `MANAGER` bisa memanggil API ini.
- **Delete(':id') (Delete):** Mengunci endpoint penghapus data dengan izin `@Permissions('user:delete')`. Berdasarkan hierarki di Part 1, hanya `SUPER_ADMIN` yang bisa memanggil API ini.

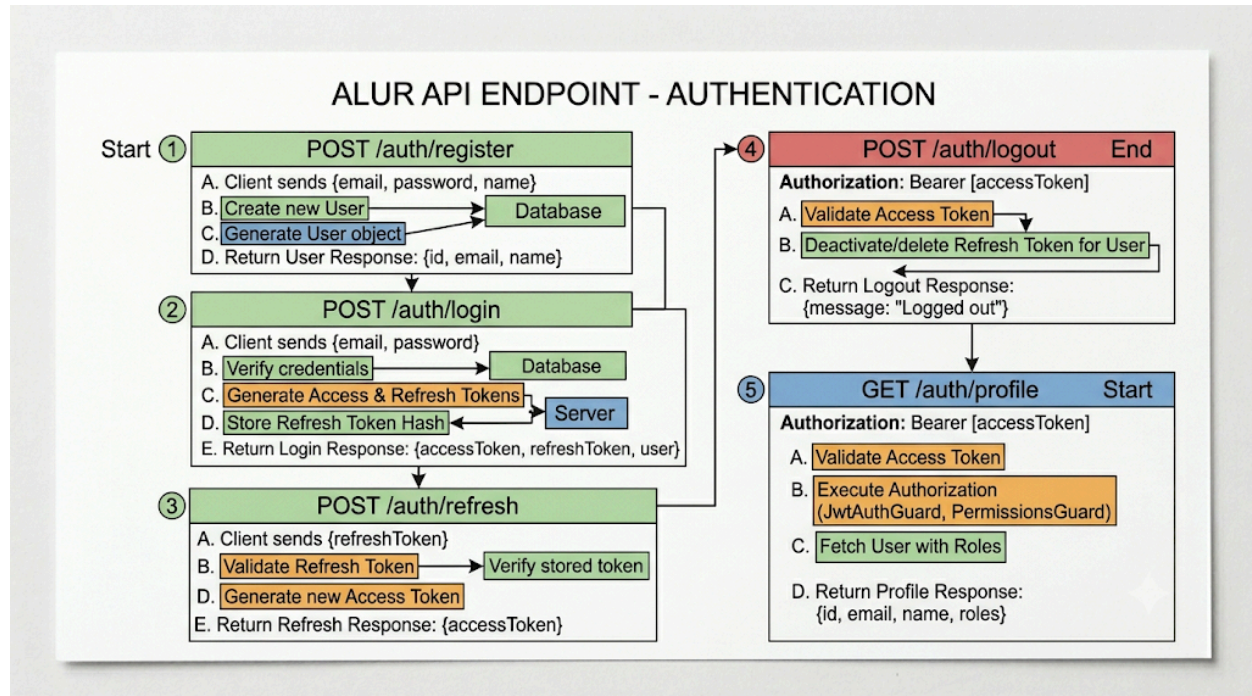
Ringkasan Alur Kerja Hak Akses di NestJS:

1. Request Masuk: `POST /users` membawa Token JWT.
2. `JwtAuthGuard`: Validasi Token. Token valid, tempel data user ke request. Akses Lolos ke Guard Berikutnya.
3. `PermissionsGuard`:
  - a. Membaca `@Permissions('user:create')` dari controller.
  - b. Ambil ID user dari request.

- c. Query database, menarik daftar Permission user tersebut (contoh: ['user:read', 'user:update']).
  - d. Mencocokkan: Apakah user:create ada di dalam daftar user? TIDAK.
  - e. Catat log akses ditolak ke AuditLog.
  - f. Kembalikan false.
4. Sistem NestJS: Mengirimkan respon 403 Forbidden ke klien. User ditolak aksesnya.

## G.ALUR API ENDPOINT

### 1.Authentication Endpoints



POST /auth/register

**Fungsi:** Daftar akun baru

**Body:** { email, password, name }

**Response:** { id, email, name }

POST /auth/login

**Fungsi:** Login dan dapat token

**Body:** { email, password }

**Response:** { accessToken, refreshToken, user }

POST /auth/refresh

**Fungsi:** Dapat access token baru

**Body:** { refreshToken }

**Response:** { accessToken }

POST /auth/logout

**Fungsi:** Logout dan hapus refresh token

Header: Authorization: Bearer [accessToken]

Response: { message: "Logged out" }

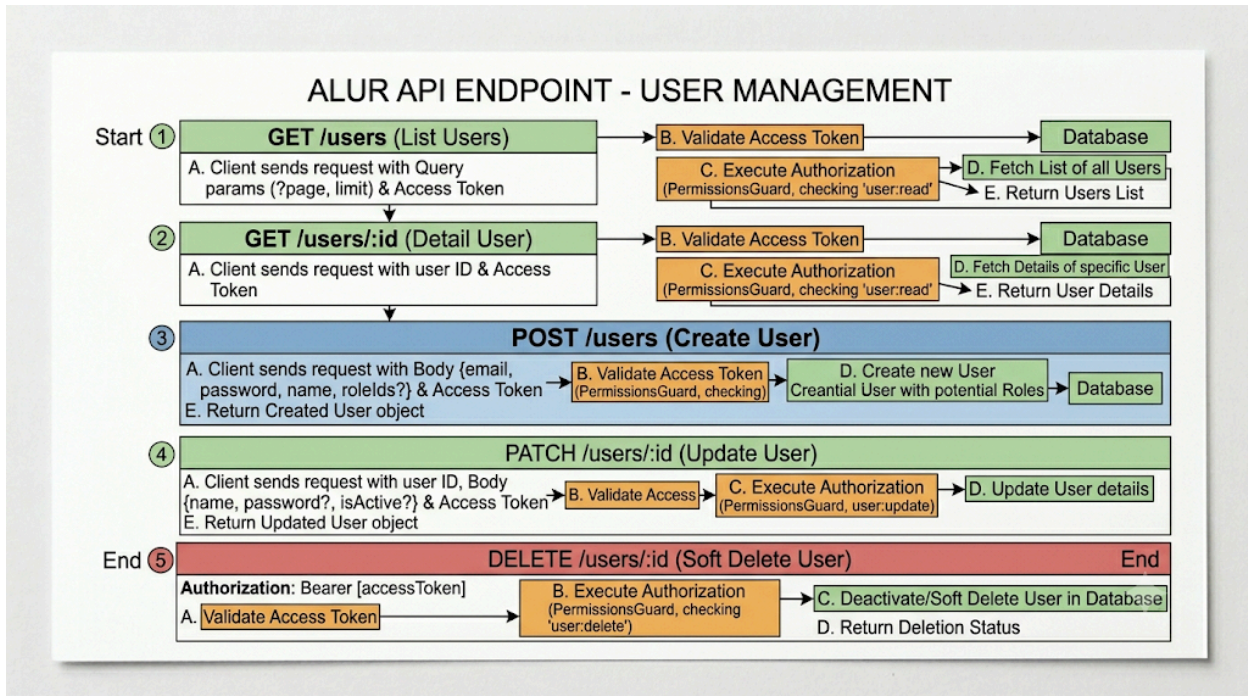
GET /auth/profile

Fungsi: Dapat data profil sendiri

Header: Authorization: Bearer [accessToken]

Response: { id, email, name, roles }

## 2. User Management Endpoints



GET /users

Fungsi: List semua user (admin only)

Header: Authorization: Bearer [accessToken]

Permission: user:read

Query: ?page=1&limit=10

GET /users/:id

Fungsi: Detail user tertentu

Header: Authorization: Bearer [accessToken]

Permission: user:read

POST /users

Fungsi: Buat user baru

Header: Authorization: Bearer [accessToken]

Permission: user:create

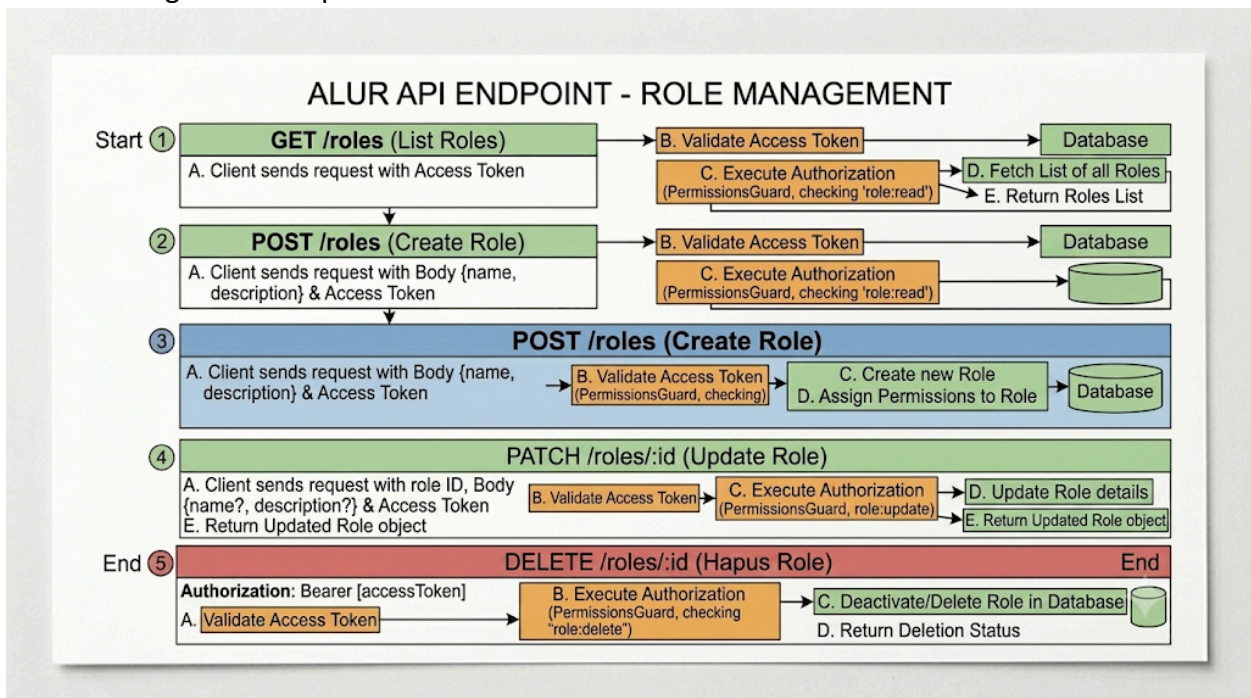
Body: { email, password, name, roleIds? }

PATCH /users/:id

**Fungsi:** Update user  
**Header:** Authorization: Bearer [accessToken]  
**Permission:** user:update  
**Body:** { name, password?, isActive? }

**DELETE /users/:id**  
**Fungsi:** Soft delete user  
**Header:** Authorization: Bearer [accessToken]  
**Permission:** user:delete

### 3.Role Management Endpoints



**GET /roles**  
**Fungsi:** List semua role  
**Permission:** role:read

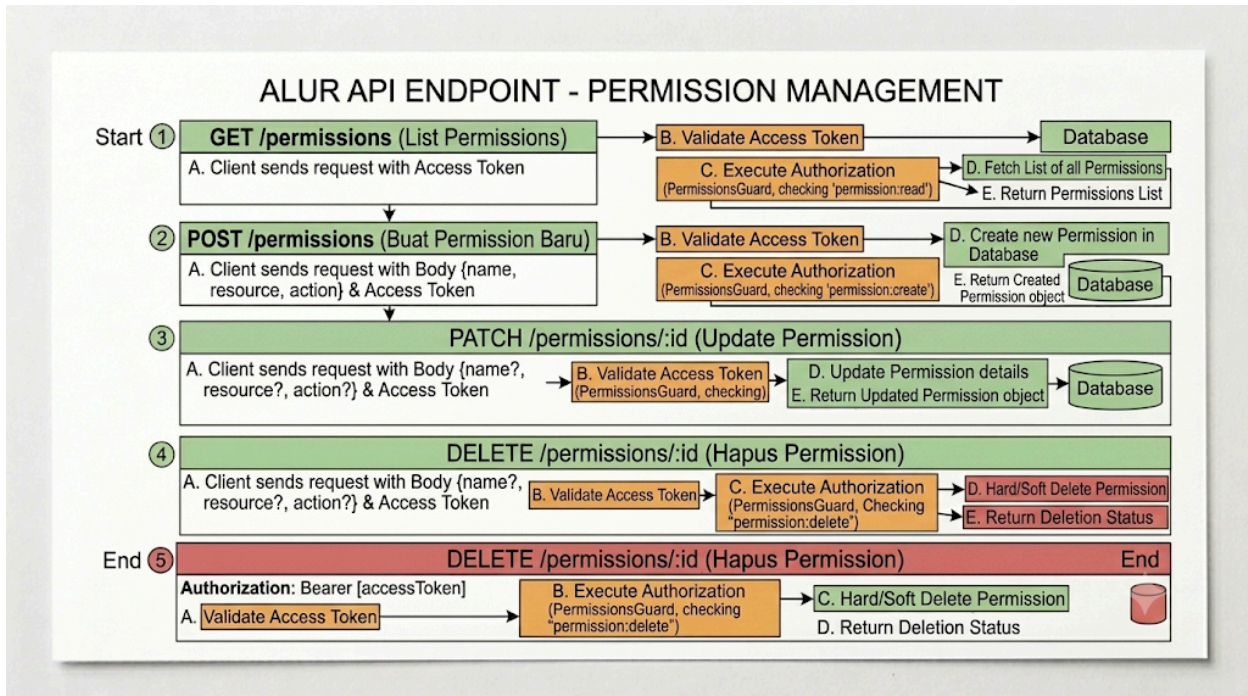
**POST /roles**  
**Fungsi:** Buat role baru  
**Permission:** role:create  
**Body:** { name, description }

**PATCH /roles/:id**  
**Fungsi:** Update role  
**Permission:** role:update  
**Body:** { name?, description? }

**DELETE /roles/:id**  
**Fungsi:** Hapus role

Permission: role:delete

#### 4. Permission Management Endpoints



GET /permissions

Fungsi: List semua permission

Permission: permission:read

POST /permissions

Fungsi: Buat permission baru

Permission: permission:create

Body: { name, resource, action }

PATCH /permissions/:id

Fungsi: Update permission

Permission: permission:update

DELETE /permissions/:id

Fungsi: Hapus permission

Permission: permission:delete

#### 5. Assignment Endpoints

POST /user-roles

Fungsi: Assign role ke user

Body: { userId, roleId }

Header: Authorization: Bearer [accessToken]

DELETE /user-roles/:id

Fungsi: Hapus assignment role

POST /role-permissions

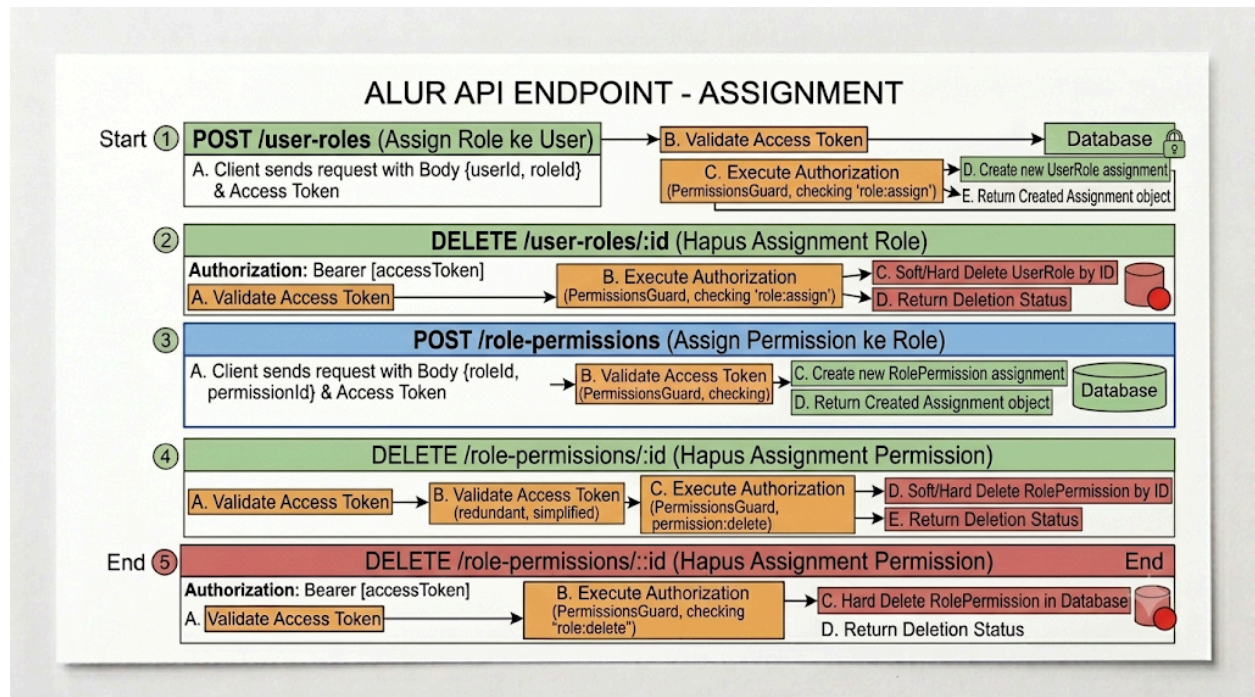
Fungsi: Assign permission ke role

Body: { roleId, permissionId }

Header: Authorization: Bearer [accessToken]

DELETE /role-permissions/:id

Fungsi: Hapus assignment permission



## H.JWT TOKEN DESIGN

Berikut adalah penjelasan mendalam mengenai arsitektur dan mekanisme JWT Token Design serta Refresh Token Flow yang kita gunakan untuk proyek enterprise ini.

### 1.Struktur Anatomi JWT (Access Token)

Access token yang kita rancang menggunakan stKitar industri dengan membagi token menjadi tiga bagian yang dipisahkan oleh Kita titik (.).

**[Header].[Payload].[Signature]**

#### a.Header (Metadata)

Menentukan algoritma kriptografi yang digunakan untuk mengunci token dan tipe token itu sendiri.

- **alg: "HS256"**: Menggunakan HMAC-SHA256, sebuah algoritma symmetric key di mana server menggunakan satu kunci rahasia (secret key) yang sama untuk membuat dan memverifikasi token.

- **typ: "JWT"**: Menegaskan bahwa objek ini adalah JSON Web Token.

### **b.Payload (Data/Claims)**

Berisi informasi kontekstual pengguna. Karena JWT dapat didekode secara bebas oleh siapa saja (bukan dienkripsi, hanya ditandatangani), jangan pernah menyimpan data sensitif seperti password di sini.

- **sub: "user-123"** (Subject): ID unik user di database Kita (menggunakan CUID). Ini dibaca oleh PermissionsGuard untuk query hak akses.
- **email**: Informasi identitas utama pengguna.
- **roles**: Menyimpan array role user (misal: ["SUPER\_ADMIN"]). Menyimpan role di payload sangat taktis untuk optimasi, karena beberapa endpoint yang hanya butuh cek role dasar tidak perlu melakukan query ulang ke database.
- **iat** (Issued At): Timestamp kapan token ini dibuat.
- **exp** (Expiration Time): Timestamp kapan token ini mati (diatur 15 menit). Waktu yang pendek ini meminimalisasi risiko jika access token sempat dicuri oleh pihak asing.

### **c.Signature (Verifikasi Keamanan)**

Bagian paling krusial untuk memastikan token tidak dimanipulasi (tampering). Server akan menggabungkan Header dan Payload yang sudah di-encode, lalu membumbunya dengan `secret_key` yang hanya diketahui oleh server backend Kita. Jika peretas mencoba mengubah isi payload (misal mengubah "roles": ["USER"] menjadi ["SUPER\_ADMIN"]), tKita tangan digitalnya menjadi tidak valid dan NestJS akan langsung melempar error 401 Unauthorized.

## **2.Mekanisme Kerja Refresh Token Flow**

Mekanisme ini dirancang agar aplikasi kita tetap aman (karena access token cepat mati) namun tetap memberikan pengalaman pengguna yang nyaman (user tidak perlu bolak-balik mengetik password setiap 15 menit).

Berikut adalah visualisasi urutan kejadian (lifecycle) saat sistem berjalan:

| Langkah | Aksi / Skenario | Detail Proses                                                                                                                                                                                                         |
|---------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | User Login      | User mengirim email & password ke POST /auth/login. Jika valid, server mengirim Access Token (hidup 15 menit) dan Refresh Token (hidup 30 hari). Di database, hash dari refresh token disimpan di model RefreshToken. |
| 2       | Request Normal  | Klien menyimpan token dan mulai menempelkannya di setiap request API pada header Authorization: Bearer [accessToken].                                                                                                 |
| 3       | Token Expired   | Setelah 15 menit, Access Token mati. Saat klien memanggil GET /api/users, JwtAuthGuard mendeteksi klaim exp telah terlewati, membatalkan request, dan mengembalikan status 401 Unauthorized.                          |
| 4       | Refresh Trigger | Frontend (misalnya menggunakan Interceptors di Axios/Fetch) menangkap error 401 tersebut. Secara otomatis di balik layar, frontend menembak endpoint POST /auth/refresh dengan membawa refreshToken di dalam body.    |

|   |                |                                                                                                                                                                                                                           |
|---|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | Token Issuance | NestJS memeriksa apakah refreshToken tersebut masih berlaku, ada di database, dan cocok dengan tokenHash. Jika lolos, server menerbitkan Access Token baru yang segar (berlaku 15 menit ke depan).                        |
| 6 | Retry Request  | Klien menerima Access Token baru, memperbarui penyimpanannya, dan langsung menembak ulang (retry) request GET /api/users yang sempat gagal tadi. User tidak menyadari proses ini karena terjadi dalam hitungan milidetik. |

Access Token : 15 menit

Refresh Token : 30 hari

Payload :

```
{
  "sub": "user-id",
  "email": "admin@test.com",
  "roles": ["SUPER_ADMIN"]
}
```

## I.PERMISSION DECORATOR

Berikut adalah penjelasan mendalam mengenai cara kerja Permission Decorator dan bagaimana PermissionsGuard mengeksekusi alur pemeriksaan tersebut hingga menghasilkan keputusan akhir (Allow/Deny).

### 1.Apa itu Permission Decorator?

Di NestJS, `@Permissions('user:create')` adalah sebuah Custom Decorator yang bertindak sebagai penKita metadata. Decorator ini tidak memblokir request secara langsung, melainkan memberi tahu framework:

**"Endpoint `POST /users` ini dikunci, dan hanya boleh dimasuki oleh siapapun yang mengantongi izin `user:create`."**

Secara internal, decorator ini menggunakan Reflector bawaan NestJS untuk menempelkan string 'user:create' ke dalam execution context metode `create()`.

### 2.Bedah Alur Kerja Guard: Role → Permission → Allow

Ketika seorang user mengeksekusi endpoint tersebut, PermissionsGuard akan bangun dan memeriksa hak akses dengan alur berantai seperti di bawah ini:

Proses Detail Langkah demi Langkah:

#### a.Lapisan User (Token Terbaca)

- JwtAuthGuard lolos → data user (termasuk ID atau sub) berhasil diekstrak dari JWT dan ditempel ke objek request.

#### b.Lapisan Role (Siapa Kita?)

- PermissionsGuard mengambil ID user tersebut, lalu melakukan query ke tabel UserRole.
- Sistem mendeteksi Role apa saja yang dimiliki user. (Contoh: User bernama "Andi" memiliki Role ADMIN).

### c.Lapisan Permission (Apa yang Bisa Kita Lakukan?)

- Dari Role ADMIN tadi, Guard melangkah ke tabel junction RolePermission untuk menarik semua action spesifik yang diikat pada role tersebut.
- Hasilnya, sistem mendapatkan daftar mentah teks izin milik Andi, misalnya: ['user:read', 'user:create', 'user:update'].

### d.Lapisan Evaluasi & Keputusan (Allow / Deny)

- Guard mengambil string yang diminta oleh decorator target, yaitu 'user:create'.
- Guard melakukan pencocokan: Apakah 'user:create' ada di dalam daftar izin milik Andi?
- ALLOW (True): Karena Andi punya izin tersebut, Guard mengembalikan nilai true dan fungsi create() di Controller dieksekusi.
- DENY (False): Jika tidak cocok (misal yang login adalah USER biasa yang hanya punya izin user:read), Guard memicu fungsi logAccessDenied() ke AuditLog, lalu melempar error 403 Forbidden.

Mengapa Alur Ini Sangat Fleksibel?

Dengan alur Role → Permission → Allow, aplikasi kita menjadi sangat dinamis.

Jika suatu hari kebijakan perusahaan berubah dan seorang **MANAGER** tiba-tiba diizinkan untuk membuat user (**user:create**), **Kita tidak perlu mengubah satu baris kode pun di NestJS**. Kita cukup masuk ke database (atau melalui dashboard admin) dan menambahkan relasi baru di tabel **RolePermission** yang menghubungkan **MANAGER** ke permission **user:create**. Endpoint Kita akan langsung mengenali perubahan tersebut secara *real-time*.

## J.ENVIRONMENT

Berikut adalah penjelasan detail mengenai file Environment Variables (.env) yang kita gunakan. File ini bertindak sebagai pusat kendali konfigurasi aplikasi kita dan akan dibaca secara terpusat oleh folder config/ yang sudah kita bahas di awal.

### A.Analisis & Fungsi Setiap Variabel

#### 1. Environment & Network

- NODE\_ENV=development
  - Fungsi: Menentukan mode berjalan aplikasi.
  - Dampaknya pada NestJS: Dalam mode development, NestJS biasanya akan mengaktifkan log yang lebih detail (verbose) dan menampilkan stack trace error yang lengkap untuk memudahkan debugging. Saat naik ke server production, variabel ini wajib diubah menjadi production demi keamanan dan performa.
- PORT=3000
  - Fungsi: Menentukan port jaringan lokal tempat server NestJS mendengarkan (listening) request masuk. Port 3000 adalah stKitar default untuk framework berbasis Node.js.

## 2. Database Connection

- DATABASE\_URL=
- Fungsi: Menyimpan string koneksi (Connection String) ke database PostgreSQL kita.
- Format yang Diperlukan Prisma: Karena kita menggunakan PostgreSQL, kita harus mengisinya dengan format berikut:

```
DATABASE_URL="postgresql://username:password@localhost:5432/nama_database?schema=public"
```

- Catatan Keamanan: Jangan pernah melakukan commit file .env yang berisi DATABASE\_URL asli ke GitHub. Gunakan .env.example sebagai templat.

## 3. JSON Web Token (JWT) Secrets

- JWT\_ACCESS\_SECRET=
  - Fungsi: Kunci rahasia (secret key) yang digunakan server untuk menandatangani (sign) Access Token. Kunci ini digunakan oleh algoritma HS256 di bagian Signature JWT.
- JWT\_REFRESH\_SECRET=
  - Fungsi: Kunci rahasia khusus yang digunakan untuk menandatangani Refresh Token.
  - Tips Taktis: Pastikan nilai JWT\_ACCESS\_SECRET dan JWT\_REFRESH\_SECRET berbeda dan diisi dengan string acak yang panjang (misalnya hasil generator base64/hex) agar tidak mudah ditebak dengan metode brute-force.

## 4. Token Expiration (Masa Berlaku)

- JWT\_ACCESS\_EXPIRES=15m
  - Fungsi: Mengatur masa kedaluwarsa Access Token selama 15 menit (m = minutes). Ini memaksa frontend untuk melakukan Refresh Token Flow secara berkala demi menjaga keamanan sesi.
- JWT\_REFRESH\_EXPIRES=30d
  - Fungsi: Mengatur masa hidup Refresh Token selama 30 hari (d = days). Selama masa ini, user tidak perlu mengetik ulang username dan password mereka, kecuali jika mereka sengaja melakukan logout atau token dihapus dari database.

## B. Alur Pemetaan ke Struktur Folder NestJS Kita

Di awal, kita memiliki folder config/. Variabel .env di atas tidak boleh dipanggil menggunakan process.env.VARIABLE secara acak di dalam Service atau Controller. Alurnya adalah sebagai berikut:

[.env File] —► Diambil oleh @nestjs/config —► Dipetakan ke config/\*.config.ts —► Dipakai di Modules

Sebagai contoh, berikut adalah bagaimana file .env kita dibaca secara rapi di dalam config/[jwt.config.ts](#):

TypeScript

```
import { registerAs } from '@nestjs/config';

export default registerAs('jwt', () => ({
  accessSecret: process.env.JWT_ACCESS_SECRET,
  refreshSecret: process.env.JWT_REFRESH_SECRET,
  accessExpires: process.env.JWT_ACCESS_EXPIRES || '15m',
  refreshExpires: process.env.JWT_REFRESH_EXPIRES || '30d',
}));
```

Dengan metode ini, jika ada salah ketik pada nama variabel lingkungan, aplikasi kita akan langsung mengetahuinya sejak awal proses bootstrapping di main.ts.

## K.DEPLOYMENT & INFRASTRUCTURE

Berikut adalah penjelasan mendalam mengenai arsitektur Deployment & Infrastructure menggunakan Docker dan Docker Compose untuk aplikasi NestJS kita yang berbasis PostgreSQL dan Prisma.

### A. Dockerfile (Cetak Biru Image Aplikasi)

#### Docker Setup

Nama file dockerfile :

```
# Dockerfile - Seperti resep membuat aplikasi
FROM node:22-alpine          # 1. Ambil base image Node.js
WORKDIR /app                 # 2. Buat folder kerja
COPY package*.json ./        # 3. Copy file dependencies
RUN npm install               # 4. Install dependencies
COPY . .                     # 5. Copy semua source code
RUN npx prisma generate       # 6. Generate Prisma client
RUN npm run build             # 7. Build aplikasi
CMD ["node", "dist/main.js"] # 8. Jalankan aplikasi
```

Dockerfile adalah sebuah berkas teks berisi instruksi berurutan untuk membangun sebuah Docker Image mandiri yang mengisolasi aplikasi NestJS kita beserta seluruh dependensinya.

Penjelasan Logika Setiap Baris (Step-by-Step):

#### 1. FROM node:22-alpine

- a. **Fungsi:** Mengambil *base image* resmi Node.js versi 22 yang berjalan di atas Alpine Linux.
- b. **Tujuan Taktis:** Alpine Linux dipilih karena ukurannya yang sangat kecil (hanya beberapa megabyte). Ini membuat hasil akhir *image* Docker kita jauh lebih ringan, cepat diunduh, dan memiliki celah keamanan minimum (*attack surface* yang kecil).

#### 2. WORKDIR /app

- a. **Fungsi:** Membuat folder direktori bernama `/app` di dalam *container* dan menentukannya sebagai ruang kerja utama untuk semua perintah berikutnya.

#### 3. COPY package\*.json ./

- a. **Fungsi:** Menyalin berkas `package.json` dan `package-lock.json` dari komputer lokal ke direktori kerja saat ini (`./`) di dalam container.

#### 4. RUN npm install

- a. **Fungsi:** Mengeksekusi instalasi seluruh dependensi (*library*) pihak ketiga yang tercantum pada berkas JSON tadi.
- b. **Optimasi Docker Cache:** Langkah 3 dan 4 diletakkan *sebelum* menyalin kode sumber. Tujuannya adalah memanfaatkan fitur *Docker Caching*. Jika kita tidak mengubah dependensi (tidak menambah library baru), Docker tidak akan mengulang proses `npm install` yang memakan waktu lama saat kita membangun ulang image, melainkan langsung menggunakan cache yang sudah ada.

#### 5. COPY ..

- a. **Fungsi:** Menyalin seluruh berkas kode sumber aplikasi NestJS kita dari komputer lokal ke dalam container.

#### 6. RUN npx prisma generate

- a. **Fungsi:** Menjalankan generator dari Prisma ORM di dalam container.
- b. **Tujuan:** Langkah ini sangat krusial agar *Prisma Client* mengenali skema database terbaru kita di dalam folder `node_modules` container, sehingga *Typescript typing* dan query database berjalan lancar saat proses kompilasi.

#### 7. RUN npm run build

- a. **Fungsi:** Mengompilasi (*compile*) kode sumber NestJS dari format TypeScript (`.ts`) yang berada di folder `src/` menjadi kode JavaScript biasa (`.js`) yang siap produksi di dalam folder `dist/`.

#### 8. CMD ["node", "dist/main.js"]

- a. **Fungsi:** Instruksi utama untuk menjalankan aplikasi saat container resmi dinyalakan. Ini mengeksekusi file entrypoint hasil kompilasi.

### B. Docker Compose (Orkestrasi Multi-Service)

Docker Compose - Menjalankan Semua Service

```
version: '3.8'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:16 # Database
```

```
    environment:
```

```
      POSTGRES_USER: ${DB_USER}
```

```
      POSTGRES_PASSWORD: ${DB_PASSWORD}
```

```
      POSTGRES_DB: ${DB_NAME}
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
    ports:
```

```
      - "5432:5432"
```

```
  api:
```

```
    build: . # Build dari Dockerfile
```

```
    environment:
```

DATABASE\_URL :

```
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
  ports:
    - "3000:3000"
  depends_on:
    - postgres

nginx:
  image: nginx:alpine # Load balancer
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
  depends_on:
    - api
```

docker-compose.yml digunakan untuk mengorquestrasi atau menjalankan banyak container sekaligus (postgres dan api) hanya dengan satu perintah: docker compose up --build.

Penjelasan Komponen Utama & Alur Jaringan:

### 1. Service **postgres** (Database Layer)

- **image: postgres:16:** Menggunakan image resmi PostgreSQL versi 16.
- **environment:** Mengambil variabel lingkungan secara dinamis dari file **.env** sistem kita untuk menjaga kerahasiaan kredensial database.
- **volumes:** - **postgres\_data:/var/lib/postgresql/data:** **Sangat Penting!** Bagian ini memastikan persistensi data. Jika container database dimatikan atau dihancurkan, seluruh data pengguna, role, permission, dan audit log kita **tidak akan hilang** karena disimpan dengan aman di dalam *volume* lokal komputer host (**postgres\_data**).
- **ports:** - **"5432:5432":** Memetakan port database internal container (**5432**) ke port komputer lokal kita (**5432**). Ini berguna jika kita ingin mengecek isi tabel database menggunakan aplikasi GUI eksternal seperti DBeaver atau TablePlus.

### 2. Service **api** (Application Layer)

- **build:** **..** Memberitahu Docker Compose untuk membangun (*build*) image aplikasi berdasarkan instruksi yang ada di berkas **Dockerfile** di direktori saat ini.
- **environment:** Menetapkan variabel **DATABASE\_URL** untuk kebutuhan internal aplikasi NestJS. Perhatikan bagian host-nya:
- Cuplikan kode

```
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
```

- Di sini Kita tidak menggunakan **localhost**, melainkan nama service database-nya, yaitu **postgres**. Docker Compose secara otomatis membuat jaringan internal (*bridge network*) khusus dan bertindak sebagai DNS Server lokal yang menerjemahkan kata **postgres** menjadi IP internal container database tersebut.
- **ports**: - **"3000:3000"**: Membuka jalur akses luar, memetakan port **3000** di dalam container NestJS ke port **3000** di komputer Kita, sehingga API kita dapat ditembak melalui Postman di alamat <http://localhost:3000>.
- **depends\_on**: - **postgres**: Menetapkan aturan hierarki urutan menyala. NestJS tidak akan dijalankan sebelum container **postgres** dalam kondisi hidup (*running*). Hal ini mencegah aplikasi NestJS mengalami *crash* di awal akibat gagal melakukan koneksi ke database yang belum siap menerima koneksi.

### C.Cara Menjalankan di Server/Lokal

Cukup jalankan satu baris perintah ini di terminal kita untuk menyalakan seluruh ekosistem aplikasi beserta sistem otorisasi RBAC-nya:

**docker compose up -d --build**

- **--build**: Memastikan Docker menyusun ulang kode aplikasi NestJS terbaru.
- **-d** (*detached mode*): Menjalankan container di latar belakang (*background*), sehingga terminal Kita tetap bisa digunakan untuk perintah lain.

### L.GITHUB

Berikut adalah panduan taktis langkah-demi-langkah untuk menginisialisasi struktur tersebut di komputer lokal kita, mengonfigurasinya, hingga mendorongnya (push) ke repositori GitHub.

#### Langkah 1: Inisialisasi Proyek NestJS di Lokal

Buka terminal kita dan jalankan perintah berikut untuk membuat proyek baru dan membentuk struktur folder yang kita inginkan:

```
# 1. Buat proyek NestJS baru (pilih npm atau yarn saat ditanya)
nest new zafi-service-app
cd zafi-service-app
```

```
# 2. Hapus file boilerplate bawaan yang tidak diperlukan di root src
rm src/app.controller.ts src/app.controller.spec.ts src/app.service.ts
```

```
# 3. Buat folder struktur dasar (Config, Common, Infrastructure, Modules, Shared, Database)
mkdir -p src/config src/common/decorators src/common/guards src/common/filters
src/common/interceptors src/common/pipes src/common/enums src/common/constants
src/common/helpers
```

```
mkdir -p src/infrastructure/prisma src/infrastructure/cache src/infrastructure/logger
mkdir -p src/modules/auth src/modules/users src/modules/roles src/modules/permissions
src/modules/role-permissions src/modules/user-roles src/modules/audit-logs
mkdir -p src/shared/dto src/shared/interfaces src/shared/types src/shared/responses
mkdir -p database/seed database/factories
```

## Langkah 2: Buat File Konfigurasi Git & Environment

Sebelum melakukan commit pertama, buat berkas-berkas esensial di root direktori proyek kita:

### 1. File .gitignore

Pastikan file sensitif dan dependensi tidak ikut terunggah. Buat atau perbarui file .gitignore kita:

```
# Dependencies
```

```
/node_modules
```

```
/.pnp
```

```
.pnp.js
```

```
# Production / Build
```

```
/dist
```

```
/build
```

```
# Environment Variables
```

```
.env
```

```
.env.production
```

```
.env.development
```

```
.env.local
```

```
# IDEs and editors
```

```
.idea/
```

```
.vscode/
```

```
*.suo
```

```
*.ntvs*
```

```
*.njsproj
```

```
*.sln
```

```
*.sw?
```

```
# Logs
```

```
npm-debug.log*
```

```
yarn-debug.log*
```

```
yarn-error.log*
```

### 2. File .env.example

Sebagai templat acuan lingkungan bagi anggota tim lain:

```
NODE_ENV=development
```

```
PORT=3000
```

```
DATABASE_URL="postgresql://username:password@localhost:5432/nama_database?schema=public"
```

```
JWT_ACCESS_SECRET="ganti_dengan_string_acak_panjang"  
JWT_REFRESH_SECRET="ganti_dengan_string_acak_panjang_lainnya"
```

```
JWT_ACCESS_EXPIRES=15m  
JWT_REFRESH_EXPIRES=30d
```

### **Langkah 3: Membuat Placeholder File Agar Folder Kosong Terbaca Git**

Secara bawaan, Git tidak akan membaca atau melacak folder yang kosong. Agar seluruh struktur folder ideal kita langsung terbentuk saat di-push ke GitHub, kita perlu membuat file placeholder (biasanya dinamakan .gitkeep) di dalam folder-folder yang belum berisi file kode:

```
touch src/config/app.config.ts src/config/database.config.ts src/config/jwt.config.ts  
src/config/swagger.config.ts  
touch src/common/decorators/roles.decorator.ts  
src/common/decorators/permissions.decorator.ts src/common/decorators/public.decorator.ts  
touch src/common/guards/jwt-auth.guard.ts src/common/guards/roles.guard.ts  
src/common/guards/permissions.guard.ts  
touch src/common/filters/http-exception.filter.ts  
touch src/common/interceptors/transform.interceptor.ts  
touch src/common/pipes/.gitkeep src/common/enums/.gitkeep  
src/common/constants/.gitkeep src/common/helpers/.gitkeep  
touch src/infrastructure/prisma/prisma.module.ts src/infrastructure/prisma/prisma.service.ts  
touch src/infrastructure/cache/.gitkeep src/infrastructure/logger/.gitkeep  
touch src/modules/auth/.gitkeep src/modules/users/.gitkeep src/modules/roles/.gitkeep  
src/modules/permissions/.gitkeep src/modules/role-permissions/.gitkeep  
src/modules/user-roles/.gitkeep src/modules/audit-logs/.gitkeep  
touch src/shared/dto/.gitkeep src/shared/interfaces/.gitkeep src/shared/types/.gitkeep  
src/shared/responses/.gitkeep  
touch database/seed/.gitkeep database/factories/.gitkeep
```

### **Langkah 4: Inisialisasi Git Lokal dan Commit Pertama**

Sekarang, mari daftarkan proyek ini ke sistem Git lokal kita dan buat snapshot pertamanya.

#### **# 1. Inisialisasi Git (jika belum otomatis saat 'nest new')**

```
git init
```

#### **# 2. Pastikan Kita berada di branch utama dengan nama 'main'**

```
git branch -M main
```

#### **# 3. Daftarkan semua file ke dalam staging area Git**

```
git add .
```

#### # 4. Lakukan commit pertama kita

**git commit -m "chore: initial commit with enterprise module-based folder structure"**

#### Langkah 5: Menghubungkan ke GitHub dan Push

- Buka browser Kita, masuk ke GitHub, dan buat sebuah repositori baru (misal dinamakan my-nestjs-rbac-app).
- Jangan centang opsi "Add a README file", "Add .gitignore", atau "Choose a license" karena kita sudah membuatnya secara manual di lokal.
- Salin URL repositori GitHub Kita :  
(contoh: <https://github.com/username/my-nestjs-rbac-app.git>).
- Kembali ke terminal komputer lokal Kita, lalu jalankan perintah berikut:

##### # 1. Hubungkan repositori lokal dengan remote repositori di GitHub

**git remote add origin <https://github.com/username/my-nestjs-rbac-app.git>**

##### # 2. Push kode kita dari branch main lokal ke GitHub

**git push -u origin main**

#### Langkah 6: Membuat Branch develop untuk Workflow Tim

Sesuai dengan strategi percabangan (Git Branching Strategy) yang sempat kita bahas untuk skala enterprise, buatlah branch develop dari main sebagai pusat integrasi fitur harian:

##### # 1. Buat branch baru bernama develop dan langsung berpindah ke sana

**git checkout -b develop**

##### # 2. Push branch develop ke GitHub kita

**git push -u origin develop**

#### M.STRATEGI PERCABANGAN Git (Git Branching Strategy)

Berikut adalah arsitektur Strategi Percabangan Git (Git Branching Strategy) yang ideal untuk mengelola siklus pengembangan proyek NestJS. Dengan memisahkan fungsionalitas ke dalam feature branches spesifik (auth, user, role, permission), tim kita dapat bekerja secara paralel tanpa takut terjadi konflik kode (code conflict) yang merusak sistem utama.

##### A.Alur & Visualisasi Git Branching Strategy

###### 1.Peran Masing-Masing Branch

###### a.Branch Utama (Long-Lived Branches)

- **main** (Production-Ready)
  - **Fungsi:** Menyimpan kode yang 100% stabil, sudah diuji, dan siap diakses oleh pengguna akhir di VPS Ubuntu.
  - **Aturan:** Tidak ada yang boleh melakukan **push** langsung ke branch ini. Pengisian kode hanya dilakukan melalui *Pull Request* (PR) formal dari branch **develop** setelah lolos *Code Review* dan *CI/CD Pipeline* (GitHub Actions).
- **develop** (Integration Layer)

- **Fungsi:** Pusat penggabungan (*integration*) dari semua fitur baru yang telah selesai dibuat.
- **Aturan:** Branch ini menjadi acuan utama bagi para *developer* untuk membuat cabang fitur baru (`git checkout -b feature/...`).

### b.Branch Fitur (Short-Lived Branches)

Branch ini dibuat secara spesifik per modul, diambil dari develop, dan akan dihapus setelah berhasil di-merge kembali ke develop.

- **feature/auth**
  - **Cakupan Kerja:** Implementasi `POST /auth/login`, `POST /auth/refresh`, pembuatan struktur JWT, penyusunan berkas `.env`, dan konfigurasi `JwtAuthGuard`.
- **feature/user**
  - **Cakupan Kerja:** Implementasi CRUD User, pembuatan skema database `User` di Prisma, integrasi *Soft Delete*, dan pembatasan endpoint menggunakan `PermissionsGuard`.
- **feature/role**
  - **Cakupan Kerja:** Implementasi CRUD Role Management (`/roles`), pembuatan tabel `Role`, serta penyusunan skema relasi *Many-to-Many* antara User dan Role.
- **feature/permission**
  - **Cakupan Kerja:** Pembuatan kustom dekorator `@Permissions()`, implementasi CRUD Permission, pengikatan *permission* ke *role* (`/role-permissions`), dan aktivasi pencatatan otomatis ke `AuditLog`.

### c.Alur Kerja Harian (Workflow Example)

Jika kita ingin mulai menggarap fitur autentikasi, berikut adalah urutan perintah Git taktis yang dilakukan di terminal:

**# 1. Pastikan kita berada di develop dan mengambil kode terbaru**

`git checkout develop`

`git pull origin develop`

**# 2. Buat branch fitur baru khusus auth**

`git checkout -b feature/auth`

**# 3. Lakukan coding (membuat JWT, Login, Refresh Token, dll)**

**# 4. Simpan perubahan kita secara berkala**

`git add .`

`git commit -m "feat(auth): implement jwt token design and refresh token flow"`

**# 5. Push branch fitur kita ke GitHub**

`git push origin feature/auth`

Setelah di-push, kita tinggal membuka halaman GitHub untuk membuat Pull Request (PR) dari feature/auth ke develop. Di sanalah GitHub Actions akan otomatis berjalan untuk memeriksa apakah kode kita aman sebelum digabungkan.

## N.SECURITY CHECKLIST

Berikut adalah penjelasan mendalam mengenai Security Checklist untuk aplikasi NestJS enterprise kita. Kombinasi 10 poin ini mencakup keamanan dari lapisan aplikasi (kode), basis data, hingga pengamanan infrastruktur server (VPS).

### A. Application & Code Security (Lapisan Aplikasi)

#### 1. Password Bcrypt (12 Rounds)

```
// Meng-hash password sebelum disimpan
const hashedPassword = await bcrypt.hash(password, 12);
// 12 rounds = cukup aman dan cepat

// Verifikasi password saat login
const isValid = await bcrypt.compare(password, user.password);
```

- **Mengapa ini penting?** Menyimpan password dalam bentuk teks biasa (*plaintext*) adalah bencana besar jika database kita bocor. Bcrypt menggunakan metode *salting* otomatis, membuat hasil hash selalu berbeda meskipun dua user menggunakan password yang sama.
- **Analisis Logika:** Angka 12 adalah faktor biaya (*work factor / rounds*). Semakin tinggi angkanya, semakin lama waktu komputasi yang dibutuhkan untuk menghasilkan atau memverifikasi hash. Angka 12 adalah standar industri saat ini: cukup berat untuk menahan serangan *Brute Force* atau *Rainbow Table*, tetapi tidak terlalu membebani CPU server kita saat melayani proses *login* normal.

#### 2. Helmet

```
// Menambahkan header keamanan
app.use(helmet());
// X-XSS-Protection: 1; mode=block
// X-Content-Type-Options: nosniff
// Strict-Transport-Security: max-age=63072000
```

- **Mengapa ini penting?** Helmet adalah middleware Node.js yang otomatis memasang belasan HTTP Security Headers pada setiap respons API kita untuk melindungi aplikasi dari kerentanan web standar.
- **Header Utama yang Dijaga:**

- **X-Content-Type-Options: nosniff:** Memaksa browser tidak menebak-nebak tipe file, mencegah serangan *MIME type sniffing* (misal: menyisipkan skrip berbahaya dalam file gambar).
- **Strict-Transport-Security (HSTS):** Memaksa browser klien untuk *hanya* berkomunikasi dengan API kita menggunakan koneksi HTTPS yang aman selama kurun waktu tertentu.

### 3. CORS Whitelist

```
app.enableCors({
  origin: ['https://app.domain.com', 'https://admin.domain.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
});
// Hanya domain tertentu yang bisa akses
```

- **Mengapa ini penting?** Secara default, browser menerapkan *Same-Origin Policy*. Tanpa konfigurasi CORS yang ketat, domain asing mana pun bisa menembak API kita melalui skrip frontend mereka.
- **Analisis Logika:** Menggunakan *whitelist* seperti ['https://app.domain.com'] memastikan bahwa hanya aplikasi frontend resmi milik kita yang diizinkan browser untuk berinteraksi dengan API. Pengaturan **credentials: true** sangat penting agar browser diizinkan mengirim dan menerima kuki (*cookies*) atau token sesi dengan aman.

### 4. Rate Limit

```
app.use(
  rateLimit({
    windowMs: 15 * 60 * 1000, // 15 menit
    max: 100 // Maks 100 request per IP
  })
);
// Mencegah serangan brute force
```

- **Mengapa ini penting?** Mencegah serangan *Denial of Service (DoS)* dan *Brute Force* pada endpoint sensitif (seperti **POST /auth/login**). Tanpa ini, penyerang bisa menembak API kita jutaan kali per detik hingga server kehabisan memori atau database kita *crash*.
- **Analisis Logika:** Aturan **max: 100** dalam waktu 15 menit per IP memastikan penggunaan API tetap berada dalam batas wajar manusia. Jika sebuah IP melebihi batas, server akan

otomatis memblokir request berikutnya dan mengembalikan status **429 Too Many Requests**.

## 5. Validation Pipe

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,           // Hanya field yang di-DTO  
    forbidNonWhitelisted: true, // Tolak field tambahan  
    transform: true           // Auto transform ke tipe yang diinginkan  
  })  
);  
// Memastikan data masuk sudah valid
```

- **Mengapa ini penting?** Ini adalah garda depan validasi tipe data sebelum masuk ke *Controller* NestJS.
- **Fungsi Opsi Taktis:**
  - **whitelist: true:** Otomatis membuang properti tambahan di luar DTO yang dikirim oleh klien.
  - **forbidNonWhitelisted: true:** Alih-alih membuang, sistem akan langsung menolak (*reject*) request dan melempar error **400 Bad Request** jika ada properti liar. Ini mencegah serangan *Mass Assignment* (penyerang mencoba menyisipkan properti `"roles": ["SUPER_ADMIN"]` saat registrasi).

## B. Token & Data Lifecycle (Lapisan Data)

### 6. Refresh Token Hash

```
// Jangan simpan refresh token plaintext  
const tokenHash = await bcrypt.hash(refreshToken, 10);  
await prisma.refreshToken.create({  
  data: {  
    userId: user.id,  
    tokenHash: tokenHash,  
    expiredAt: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000)  
  }  
});
```

- **Mengapa ini penting?** Berbeda dengan *Access Token* yang bersifat *stateless*, *Refresh Token* disimpan di database untuk validasi jangka panjang. Jika peretas berhasil

menembus database PostgreSQL kita dan mengambil *Refresh Token plaintext*, mereka bisa menggunakannya untuk terus-menerus menerbitkan *Access Token* baru.

- **Analisis Logika:** Dengan melakukan hashing (`bcrypt.hash(refreshToken, 10)`), server hanya menyimpan bukti verifikasinya saja. Bahkan jika database bocor, peretas tidak bisa menggunakan string hash tersebut untuk meminta akses ke API.

## 7. Soft Delete

```
// Jangan hapus data, tapi tandai sebagai dihapus
```

```
await prisma.user.update({  
  where: { id: userId },  
  data: { deletedAt: new Date() }  
});
```

```
// Saat query, filter yang tidak dihapus
```

```
where: { deletedAt: null }
```

- **Mengapa ini penting?** Menghapus data secara permanen (`DELETE FROM`) memiliki risiko tinggi jika terjadi kesalahan operasional atau peretasan. Selain itu, data yang dihapus permanen akan merusak relasi historis pada tabel *Audit Log*.
- **Analisis Logika:** Menambahkan kolom `deletedAt` menjaga integritas data. Di level Prisma, kita memanfaatkan *extension* global agar setiap operasi pencarian otomatis melakukan filter `where: { deletedAt: null }`, sehingga data yang "dihapus" tidak akan pernah muncul di API, namun tetap aman di database untuk kebutuhan forensik data jika diperlukan.

## C. Infrastructure & Network Security (Lapisan Server & VPS)

### 8. HTTPS (Let's Encrypt)

```
# Gunakan Let's Encrypt untuk SSL gratis
```

```
sudo certbot --nginx -d api.yourdomain.com
```

- **Mengapa ini penting?** Tanpa HTTPS, seluruh lalu lintas data antara frontend dan backend (termasuk *password* plaintext saat login dan *JWT Token* di header) dikirim dalam bentuk teks biasa. Data tersebut bisa dengan mudah disadap di tengah jalan menggunakan teknik *Man-in-the-Middle (MitM)*.
- **Analisis Logika:** `certbot` mengotomatisasi pembuatan sertifikat SSL gratis dari Let's Encrypt dan langsung mengonfigurasi Nginx untuk melakukan enkripsi asimetris secara ketat pada port 443.

## 9. Fail2Ban & UFW Firewall

```
# Install fail2ban untuk blokir IP jahat
sudo apt install fail2ban
```

```
# Setup firewall
sudo ufw allow 22/tcp # SSH
sudo ufw allow 443/tcp # HTTPS
sudo ufw allow 80/tcp # HTTP
sudo ufw enable
```

- **Mengapa ini penting?** Server VPS Ubuntu yang terekspos ke internet akan terus-menerus dipindai oleh *botnet* jahat yang mencoba menebak password SSH kita.
- **Analisis Logika:**
  - **UFW (Uncomplicated Firewall):** Bertindak sebagai satpam port. UFW menutup seluruh port VPS kecuali yang diizinkan secara eksplisit (Port 22 untuk remote SSH admin, Port 80 & 443 untuk lalu lintas web publik).
  - **Fail2Ban:** Bekerja dengan memantau log autentikasi sistem. Jika ada IP yang gagal login SSH berkali-kali dalam waktu singkat, Fail2Ban akan otomatis memperbarui aturan firewall untuk memblokir IP tersebut selama beberapa jam/hari.

## 10. PostgreSQL Tidak Expose Publik

```
# docker-compose.yml
services:
  postgres:
    ports:
      - "127.0.0.1:5432:5432" # Hanya localhost yang bisa akses
```

- **Mengapa ini penting?** Port database (5432) tidak boleh bisa diakses langsung dari internet publik, karena meningkatkan risiko serangan eksploitasi celah keamanan database atau *credential stuffing*.
- **Analisis Logika:** Pada file `docker-compose.yml`, mengubah pemetaan port menjadi "127.0.0.1:5432:5432" mengunci jaringan PostgreSQL agar hanya menerima koneksi dari dalam mesin VPS itu sendiri (*localhost*). Aplikasi NestJS kita yang berada di kontainer sebelah tetap bisa terhubung dengan aman menggunakan nama jaringan internal Docker Compose, sementara pihak luar dari internet tidak akan pernah bisa mendeteksi adanya port database yang terbuka.

## O.SWAGGER

Dokumentasi Swagger (/api/docs) menjabarkan secara rinci seluruh struktur backend kita: nama-nama endpoint, parameter query, struktur request body (DTO), jenis skema database, hingga aturan hak akses (permissions).

Jika skema ini terekspos di Production, aktor jahat dapat memanfaatkannya sebagai peta jalan (road map) untuk mencari celah keamanan (misal: mencari endpoint yang lupa dipasang PermissionsGuard atau mencoba melakukan mass assignment injection). Oleh karena itu, Swagger hanya boleh aktif di lingkungan internal: Development dan Staging (untuk keperluan testing tim frontend/QA).

### Implementasi Kode di src/main.ts

Untuk mewujudkan aturan ini, kita bisa memanfaatkan variabel lingkungan NODE\_ENV yang dibaca via konfigurasi @nestjs/config atau langsung melalui process.env. Berikut adalah cara implementasi taktisnya:

#### TypeScript

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { ConfigService } from '@nestjs/config';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // 1. Ambil ConfigService untuk membaca environment secara aman
  const configService = app.get(ConfigService);
  const nodeEnv = configService.get<string>('app.nodeEnv') || process.env.NODE_ENV;

  // 2. Kondisional Global: Hanya jalankan Swagger jika BUKAN production
  if (nodeEnv !== 'production') {
    const config = new DocumentBuilder()
      .setTitle('Enterprise RBAC API')
      .setDescription('Dokumentasi API untuk Sistem Manajemen User & Hak Akses')
      .setVersion('1.0')
      .addBearerAuth() // Mendukung pengujian token JWT langsung dari Swagger UI
      .build();

    const document = SwaggerModule.createDocument(app, config);

    // Setup endpoint di /api/docs
    SwaggerModule.setup('api/docs', app, document, {
      swaggerOptions: {
        persistAuthorization: true, // Token JWT tidak hilang saat halaman di-refresh
      },
    });
  }
}
```

```
const port = configService.get<number>('app.port') || 3000;
await app.listen(port);
}
bootstrap();
```

### Strategi Pengamanan Ekstra untuk Lingkungan Staging

Di lingkungan Staging, server biasanya sudah menggunakan IP publik atau domain yang bisa diakses via internet (misal: staging.api.yourdomain.com/api/docs). Agar orang luar tetap tidak bisa mengintip Swagger Staging Kita, terapkan salah satu dari dua metode pengaman tingkat lanjut (advanced security) berikut:

Opsi A: Mengunci Swagger dengan Basic Authentication (Rekomendasi)

Kita bisa memasang username dan password sederhana khusus untuk mengakses halaman /api/docs. Pasang library express-basic-auth di proyek NestJS kita:

```
import * as basicAuth from 'express-basic-auth';

// Letakkan tepat di atas baris SwaggerModule.setup di main.ts
if (nodeEnv === 'staging') {
  app.use(
    ['/api/docs', '/api/docs-json'],
    basicAuth({
      challenge: true,
      users: {
        // Ambil dari file .env agar tidak hardcoded
        [process.env.SWAGGER_USER]: process.env.SWAGGER_PASSWORD,
      },
    }),
  );
}
```

Opsi B: Memblokir Akses di Level Nginx (VPS Ubuntu)

Jika kita tidak ingin menyentuh kode aplikasi, kita bisa memaksa Nginx di VPS Staging untuk menolak siapa pun yang mengakses /api/docs, kecuali mereka yang berasal dari IP kantor atau IP VPN tim pengembang kita.

Tambahkan blok aturan ini pada konfigurasi berkas situs Nginx kita (/etc/nginx/sites-available/default):

```
Nginx
location /api/docs {
  allow 192.168.1.0/24; # Contoh IP internal / VPN Kantor
  allow 111.222.333.444; # Contoh IP Statis tim Lead Developer
  deny all;           # Blokir sisa IP lain dari internet publik
```

```
proxy_pass http://localhost:3000; # Teruskan ke NestJS jika IP cocok
# ... konfigurasi proxy_set_header lainnya ...
}
```

## **P.VPS Produksi**

Berikut adalah analisis taktis alokasi sumber daya (resource allocation) dan mengapa spesifikasi tersebut sangat ideal untuk arsitektur NestJS + PostgreSQL + Prisma kita.

### **1. Alokasi VPS Standard (2 vCPU, 4 GB RAM, 80 GB SSD)**

Target: 1.000–5.000 Aktif User (Bukan Concurrent)

Pada skenario ini, seluruh komponen (API, Database, Nginx) biasanya diletakkan dalam satu mesin VPS yang sama menggunakan Docker Compose untuk menghemat biaya operasional.

- Alokasi RAM (Total 4 GB):
  - OS Ubuntu Server & Nginx: ~500 MB.
  - PostgreSQL (16/17): ~1 GB sampai 1.5 GB. (Sangat disarankan mengatur `shared_buffers` di PostgreSQL ke 25% dari total RAM, yaitu sekitar 1 GB, agar query pencarian Role, Permission, dan Audit Log berjalan langsung dari memori tanpa membebani SSD).
  - NestJS App (1 Instance / PM2 Cluster 1): ~300 MB - 500 MB saat menangani trafik.
  - Sisa RAM (~1 GB): Bertindak sebagai buffer untuk OS melakukan caching I/O file dan menangani lonjakan (spike) trafik mendadak.
- Alokasi Storage (80 GB SSD):
  - Sangat aman. Kode aplikasi dan Docker image NestJS paling hanya memakan tempat 1–2 GB. Sisa kapasitas yang besar diorientasikan untuk pertumbuhan tabel AuditLog yang dipastikan akan membengkak seiring waktu karena mencatat setiap aktivitas write dan percobaan access denied.

### **2. Alokasi VPS SaaS (4 vCPU, 8 GB RAM, 160 GB SSD)**

Target: Aplikasi Multi-Tenant dengan Trafik Padat & Kompleks

Untuk arsitektur SaaS, beban komputasi akan melonjak karena sistem harus menangani isolasi data antar-tenant, enkripsi ganda, serta pencatatan audit log yang jauh lebih masif.

- Pemanfaatan vCPU (4 Core):
  - Kita bisa mengaktifkan PM2 Cluster Mode dengan kapasitas 2 atau 3 instances. NestJS (Node.js) secara bawaan bersifat single-threaded. Dengan spesifikasi 4 vCPU, Anda bisa mendistribusikan beban kerja ke 3 thread terpisah secara paralel. Nginx akan bertindak sebagai Load Balancer lokal yang membagi trafik ke tiap kluster NestJS tersebut.
- Alokasi RAM (8 GB):
  - Sangat direkomendasikan untuk menaikkan konfigurasi connection pool pada Prisma dan PostgreSQL karena jumlah koneksi simultan dari berbagai tenant akan meningkat.

- Kita juga memiliki ruang yang cukup jika di kemudian hari ingin menambahkan satu kontainer Redis Cache di dalam VPS ini untuk memotong jalur query RBAC (menyimpan data Role & Permission user di memori cache agar tidak perlu bolak-balik menembak PostgreSQL).
- Alokasi Storage (160 GB SSD):
  - Selain untuk menampung basis data multi-tenant yang besar, ruang ini krusial untuk menyimpan rolling logs (riwayat log harian dari PM2/Docker) serta kebutuhan database backup harian (pg\_dump) yang tersimpan lokal sebelum diunggah ke cloud storage eksternal (seperti AWS S3).

Jika kita menggunakan penyedia VPS seperti DigitalOcean, Linode, Hetzner, atau Biznet Gio:

1. Pilih Jenis CPU yang Tepat: Untuk VPS Standard, tipe Shared CPU sudah cukup. Namun untuk varian SaaS, sangat disarankan memilih Dedicated CPU (General Purpose / Optimized) agar performa thread Node.js tidak terganggu oleh pengguna lain dalam satu server fisik (noisy neighbors).
2. Gunakan Swap Memory: Di Ubuntu 24.04, selalu buat file Swap sebesar 2–4 GB sebagai jaring pengaman. Jika RAM fisik mendadak penuh karena lonjakan trafik, server tidak akan langsung crash atau mengalami Out-Of-Memory (OOM) Killer, melainkan melambat sejenak memanfaatkan ruang SSD tersebut.

## Q.KEAMANAN APLIKASI

**Cheat Sheet Keamanan (Security Checklist)** yang siap kita cetak atau simpan untuk memastikan aplikasi NestJS enterprise kita tidak memiliki celah keamanan, mulai dari lapisan kode hingga konfigurasi infrastruktur di VPS Ubuntu.

### I. Application & Code Level (NestJS)

| Komponen Security | Status / Konfigurasi            | Fungsi Utama                                                                                                                                        |
|-------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Password Hashing  | <code>bcrypt</code> (12 Rounds) | Mengamankan password user di database dengan algoritma <i>salting</i> yang berat untuk menahan serangan <i>Brute Force</i> / <i>Rainbow Table</i> . |
| Helmet            | <code>app.use(helmet())</code>  | Otomatis menyuntikkan HTTP Security Headers (seperti <code>X-Content-Type-Options: nosniff</code> )                                                 |

|                                  |                                                                                 |                                                                                                                                                                                      |
|----------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  |                                                                                 | untuk mencegah serangan web standar.                                                                                                                                                 |
| <b>CORS Whitelist</b>            | Eksplisit Domain                                                                | Membatasi akses API agar hanya bisa ditembak oleh domain frontend resmi Anda ( <a href="https://app.domain.com">https://app.domain.com</a> ), mencegah serangan skrip lintas domain. |
| <b>Rate Limit</b>                | Maks 100 req / 15 menit / IP                                                    | Mencegah serangan DDoS dan pemboman request pada endpoint sensitif seperti <a href="#">/auth/login</a> .                                                                             |
| <b>Validation Pipe &amp; DTO</b> | <a href="#">whitelist: true</a> ,<br><a href="#">forbidNonWhitelisted: true</a> | Membaca dekorator <a href="#">@IsEmail()</a> , <a href="#">@IsString()</a> di DTO. Membuang dan menolak request yang membawa properti liar (mencegah <i>Mass Assignment</i> ).       |

## 2.Token & Data Management Level (Prisma & PostgreSQL)

| Komponen Security         | Status Konfigurasi /               | Fungsi Utama                                                                                                                                       |
|---------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Refresh Token Hash</b> | <a href="#">bcrypt</a> (10 Rounds) | Menyimpan <i>hash</i> dari Refresh Token di database (bukan teks biasa), sehingga jika database bocor, peretas tetap tidak bisa mencuri sesi user. |

|                    |                                         |                                                                                                                                                                          |
|--------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Audit Log</b>   | Prisma<br>Middleware /<br>Extension     | Mencatat setiap aktivitas perubahan data ( <b>CREATE</b> , <b>UPDATE</b> , <b>DELETE</b> ) dan percobaan akses ilegal ( <b>ACCESS_DENIED</b> ) untuk keperluan forensik. |
| <b>Soft Delete</b> | Kolom <b>deletedAt</b><br>Global Filter | Tidak menghapus data dari disk secara permanen. Menjaga integritas data historis pada Audit Log dan mencegah kesalahan operasional fatal.                                |

### 3.Infrastructure & Network Level (VPS Ubuntu & Docker)

| Komponen Security   | Status Konfigurasi /               | Fungsi Utama                                                                                                                                         |
|---------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HTTPS (SSL)</b>  | Let's Encrypt ( <b>certbot</b> )   | Menkripsi seluruh lalu lintas data antara klien dan server (Nginx Port 443) agar tidak bisa disadap di jaringan publik ( <i>Man-in-the-Middle</i> ). |
| <b>Fail2Ban</b>     | Aktif pada Service SSH             | Memantau log sistem dan otomatis memblokir IP asing yang mencoba menebak password SSH server VPS Anda berkali-kali.                                  |
| <b>UFW Firewall</b> | <b>allow 22, 80, 443, deny all</b> | Menutup seluruh pintu/port masuk ke VPS Ubuntu, kecuali port SSH, HTTP, dan HTTPS.                                                                   |

|                             |                                  |                                                                                                                                  |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>PostgreSQL Isolation</b> | Bind ke<br><b>127.0.0.1:5432</b> | Mengunci port database di Docker Compose agar hanya bisa diakses secara lokal oleh NestJS, dan tidak terekspos ke internet luar. |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|

### Alur Pertahanan Berlapis (Defense in Depth)

Saat peretas mencoba menyerang sistem kita dari luar, mereka harus menembus 3 lapisan pertahanan ini sebelum bisa menyentuh data sensitif kita:

1. **Lapisan Jaringan (VPS):** Dijaga oleh **UFW**, **Fail2Ban**, dan **Nginx (HTTPS)**. Port database terkunci rapat.
2. **Lapisan Aplikasi (NestJS):** Dijaga oleh **Helmet**, **Rate Limit**, **CORS**, dan **Validation Pipe**. Request ilegal langsung dihempaskan sebelum menyentuh Controller.
3. **Lapisan Data (Database):** Dijaga oleh **Bcrypt Hashing**, **Soft Delete**, dan **Audit Log**. Setiap gerak-gerik manipulasi data terekam abadi.

### [Form security compliance checklist](#)

#### Ringkasan Isu Kendali & Prioritas Audit

##### 1. Lapisan Aplikasi & Kode (Application Layer)

- Password Hashing (Kritis): Algoritma bcrypt dengan 12 rounds sebagai standar industri penangkal serangan rainbow tables.
- HTTP Security Headers (Tinggi): Pemasangan middleware Helmet secara global untuk menangani mitigasi Clickjacking dan MIME sniffing.
- CORS Whitelist (Tinggi): Pelarangan penggunaan wildcard \* pada lingkungan produksi; pendefinisian domain asal wajib bersifat eksplisit.
- Rate Limiting (Tinggi): Pembatasan trafik per IP untuk mengamankan siklus hidup endpoint otentikasi dari serangan brute force.
- Validation Pipe & DTO (Kritis): Pengaktifan parameter whitelist: true dan forbidNonWhitelisted: true secara global guna memblokir celah manipulasi parameter (Mass Assignment).

##### 2. Manajemen Token & Siklus Data (Data Lifecycle)

- Refresh Token Hash (Kritis): Penyimpanan token jangka panjang di database wajib melalui enkripsi/hashing (tidak boleh berwujud plaintext).
- Granular RBAC Guard (Kritis): Verifikasi berlapis yang menggabungkan validasi hak akses berbasis Role dan Permission dinamis.
- Automated Audit Log (Tinggi): Pencatatan mutasi data fisik (CREATE, UPDATE, DELETE) dan status akses ilegal (ACCESS\_DENIED) secara otomatis.
- Soft Delete Pattern (Sedang): Penerapan kolom penanda penghapusan (deletedAt) untuk menjamin rekonsiliasi data historis saat proses forensik.

### 3. Infrastruktur & Jaringan (Infrastructure Layer)

- Transport Layer Security / HTTPS (Kritis): Penggunaan sertifikat SSL aktif (Let's Encrypt) dengan konfigurasi TLS minimum versi 1.2 atau 1.3 di level Nginx.
- Host Intrusion Defense (Tinggi): Instalasi Fail2Ban untuk mengunci otomatis IP luar yang mencoba melakukan brute force port SSH server.
- UFW Firewall Rules (Kritis): Penutupan akses port VPS secara masif dan hanya membuka jalur masuk esensial (Port 22, 80, 443).
- PostgreSQL Isolation (Kritis): Penguncian port database (5432) melalui pemetaan loopback internal (127.0.0.1) agar tidak terekspos ke internet publik.
- Swagger Restrictions (Tinggi): Isolasi rute /api/docs secara kondisional agar dinonaktifkan secara absolut pada lingkungan produksi publik.

## R.SPRINT PLAN

### Sprint 1: Setup Dasar

**Tujuan utama dari sprint ini** adalah membangun fondasi aplikasi yang kokoh, mengkonfigurasi koneksi database terisolasi, dan memastikan seluruh tim memiliki lingkungan pengembangan (development environment) yang seragam menggunakan Docker.

#### Timeline & Goal Sprint 1

- Durasi: 1–2 Hari Kerja
- Target Utama: Aplikasi NestJS menyala di dalam container Docker dan sukses terhubung ke database PostgreSQL lokal tanpa kendala network.

### Panduan Langkah Demi Langkah (Step-by-Step)

#### 1. Inisialisasi Project NestJS

Jalankan perintah ini untuk membuat boilerplate project baru dan masuk ke direktori utama:

```
nest new zafi-service-app
cd zafi-service-app
```

(Pilih package manager , disarankan menggunakan npm untuk keselarasan tim).

#### 2. Instalasi Dependensi Enterprise

Instal seluruh library yang dibutuhkan untuk kebutuhan konfigurasi, keamanan, ORM, dan otentikasi JWT:

# Dependensi Inti NestJS & Keamanan

```
npm install @nestjs/config @nestjs/jwt @nestjs/passport passport passport-jwt bcrypt
class-validator class-transformer
```

# Dependensi Prisma ORM

```
npm install @prisma/client
npm install -D prisma
```

# Dependensi Typescript (Development)

```
npm install -D @types/passport-jwt @types/bcrypt
```

### 3. Inisialisasi & Konfigurasi Skema Prisma

Jalankan perintah inisialisasi untuk membuat folder prisma/ dan berkas .env:

```
npx prisma init
```

Buka berkas prisma/schema.prisma yang baru terbentuk, lalu pastikan datasource diarahkan ke PostgreSQL:

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```
generator client {  
  provider = "prisma-client-js"  
}
```

// Catatan: Definisikan model User, Role, Permission di Sprint berikutnya.

### 4. Kontainerisasi Ekosistem (Docker Setup)

Untuk memastikan arsitektur infastruktur kita berjalan mulus, buat dua berkas konfigurasi di *root* direktori proyek kita:

#### Berkas **Dockerfile**:

```
FROM node:22-alpine  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
RUN npx prisma generate  
RUN npm run build  
CMD ["node", "dist/main.js"]
```

Berkas docker-compose.yml:

```
version: '3.8'  
services:  
  postgres:  
    image: postgres:16  
    environment:  
      POSTGRES_USER: root  
      POSTGRES_PASSWORD: secretpassword  
      POSTGRES_DB: rbac_db  
    volumes:  
      - postgres_data:/var/lib/postgresql/data
```

ports:

- "127.0.0.1:5432:5432" # Sesuai Security Checklist: Terbuka di localhost

api:

build: .

environment:

NODE\_ENV: development

PORT: 3000

DATABASE\_URL: postgresql://root:secretpassword@postgres:5432/rbac\_db?schema=public

JWT\_ACCESS\_SECRET: supersecretaccesskey123

JWT\_REFRESH\_SECRET: supersecretrefreshkey123

JWT\_ACCESS\_EXPIRES: 15m

JWT\_REFRESH\_EXPIRES: 30d

ports:

- "3000:3000"

depends\_on:

- postgres

volumes:

postgres\_data:

## 5. Eksekusi Lingkungan Pengembangan

Perbarui berkas .env lokal Kita agar selaras dengan koneksi database di dalam Docker Compose:

```
DATABASE_URL="postgresql://root:secretpassword@localhost:5432/rbac_db?schema=public"
```

Nyalakan seluruh *service* di latar belakang menggunakan perintah:

**docker compose up -d --build**

Pengecekan Kesehatan Sistem (Health Check)

Setelah container menyala, lakukan verifikasi mandiri untuk memastikan Sprint 1 selesai dengan sempurna:

- Periksa Container: Jalankan `docker ps`. Pastikan container `rbac-api-api-1` dan `rbac-api-postgres-1` berstatus Up.
- Periksa API: Buka browser dan akses `http://localhost:3000`. Pastikan muncul tulisan "Hello World!".
- Periksa Database: Coba lakukan test koneksi ke `localhost:5432` menggunakan TablePlus atau DBeaver dengan kredensial yang sesuai di berkas .env.

Jika ketiga poin di atas lolos, fondasi aplikasi kita telah siap 100% untuk masuk ke Sprint 2: Skema Database & Migrasi (Prisma)

## Sprint 2: Skema Database & Migrasi (Prisma).

Pada tahapan ini, kita akan merancang arsitektur basis data relasional di PostgreSQL menggunakan Prisma ORM. Relasi yang dibentuk bersifat hibrida: menghubungkan **User → Role** via tabel junction **UserRole**, dan **Role → Permission** via tabel junction **RolePermission**. Kita juga mengintegrasikan sistem **Soft Delete** (**deletedAt**) dan **Audit Log** secara struktural.

### Timeline & Goal Sprint 2 (Database)

- Durasi: 2 Hari Kerja
- Target Utama: Berkas `schema.prisma` terdefinisi dengan relasi yang solid, sukses melakukan migrasi fisik ke PostgreSQL, dan Prisma Client tergenerasi dengan sempurna.

### 1. Arsitektur Relasi Basis Data (ERD)

Sebelum menulis kode skema, berikut adalah visualisasi hubungan antar-tabel yang akan kita bangun di dalam PostgreSQL:

### 2. Menyusun Berkas `prisma/schema.prisma`

Buka berkas `prisma/schema.prisma` di proyek, lalu gantikan isinya dengan skema enterprise di bawah ini. Kita menggunakan format ID berupa string CUID demi performa indeksasi dan keamanan (tidak mudah ditebak seperti auto-increment integer).

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  password    String
  name        String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  deletedAt   DateTime? // Mendukung Soft Delete Checklist

  // Relasi
  userRoles   UserRole[]
  refreshTokens RefreshToken[]
  auditLogs   AuditLog[]

  @@map("users")
}
```

```

model Role {
    id      String      @id @default(cuid())
    name    String      @unique // Contoh: SUPER_ADMIN, MANAGER
    description String?
    createdAt DateTime   @default(now())
    updatedAt DateTime   @updatedAt
    deletedAt DateTime?  // Mendukung Soft Delete Checklist

    // Relasi
    userRoles    UserRole[]
    rolePermissions RolePermission[]

    @@map("roles")
}

model Permission {
    id      String      @id @default(cuid())
    name    String      @unique // Contoh: user:create, permission:read
    resource String      // Contoh: user, permission
    action   String      // Contoh: create, read, update, delete
    createdAt DateTime   @default(now())
    updatedAt DateTime   @updatedAt

    // Relasi
    rolePermissions RolePermission[]

    @@map("permissions")
}

// Tabel Junction / Pivot: Hubungan Many-to-Many (User <-> Role)
model UserRole {
    id      String @id @default(cuid())
    userId  String
    roleId  String
    createdAt DateTime @default(now())

    // Relasi formal
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)

    @@unique([userId, roleId]) // Mencegah duplikasi assignment role yang sama ke user
    @@map("user_roles")
}

```

```

// Tabel Junction / Pivot: Hubungan Many-to-Many (Role <-> Permission)
model RolePermission {
  id      String @id @default(cuid())
  roleId  String
  permissionId String
  createdAt DateTime @default(now())

  // Relasi formal
  role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)
  permission Permission @relation(fields: [permissionId], references: [id], onDelete: Cascade)

  @@unique([roleId, permissionId]) // Mencegah duplikasi assignment permission yang sama ke
  role
  @@map("role_permissions")
}

// Tabel Penyimpanan Sesi Refresh Token (Hashed)
model RefreshToken {
  id      String @id @default(cuid())
  userId  String
  tokenHash String // Sesuai Security Checklist: Disimpan sebagai hash bcrypt
  expiresAt DateTime
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@map("refresh_tokens")
}

// Tabel Audit Log Independen
model AuditLog {
  id      String @id @default(cuid())
  userId  String? // Nullable jika aksi dilakukan oleh sistem anonim/sebelum login
  action  String // Contoh: USER_CREATE, ACCESS_DENIED, ROLE_UPDATE
  endpoint String // Contoh: PATCH /permissions/:id
  ipAddress String?
  userAgent String?
  beforeData Json? // Snapshot data lama sebelum diubah
  afterData  Json? // Snapshot data baru setelah diubah
  status    String // Contoh: SUCCESS, FAILED, ACCESS_DENIED
  createdAt DateTime @default(now())

  user User? @relation(fields: [userId], references: [id], onDelete: SetNull)
}

```

```
  @@map("audit_logs")  
}
```

### 3. Eksekusi Migrasi Fisik ke PostgreSQL

Pastikan container Docker database kita dalam kondisi menyala (docker compose up -d postgres). Jalankan runtunan perintah ini di terminal lokal:

```
# 1. Jalankan proses migrasi dan buat tabel fisik di PostgreSQL  
npx prisma migrate dev --name init_rbac_architecture
```

```
# 2. Opsional: Pastikan client pengetikan Typescript di-generate ulang  
npx prisma generate
```

**Catatan Taktis:** Perintah prisma migrate dev secara otomatis akan membaca berkas .env lokal Anda, membuat histori migrasi di dalam folder prisma/migrations/, menyelaraskan struktur tabel di database PostgreSQL, serta memperbarui folder node\_modules/.prisma/client agar editor kode (VS Code) Anda mengenali semua tipe data model yang baru.

#### Verifikasi Hasil Migrasi via Prisma Studio

Untuk memastikan seluruh tabel (users, roles, permissions, user\_roles, role\_permissions, refresh\_tokens, audit\_logs) telah terbentuk tanpa cacat di dalam PostgreSQL, jalankan alat bantu visual bawaan Prisma:

```
npx prisma studio
```

Buka browser di alamat <http://localhost:5555>. Jika seluruh tabel sudah muncul di dashboard Prisma Studio, selamat! Sprint 2 bagian Database selesai dengan status Lolos Kualifikasi. Kita siap melanjutkan ke pembuatan logika inti di dalam kode aplikasi.

### Sprint 3: Otorisasi & Logika Bisnis 1 (Core Implementation **PermissionsGuard** dan decorator)

Di sini kita akan menghidupkan sistem dengan membangun komponen keamanan, pengondisian token, serta custom decorators yang akan menjadi otak dari kendali akses aplikasi kita.

#### Timeline & Goal Sprint 3

- Durasi: 3–4 Hari Kerja
- Target Utama: Menyelesaikan modul autentikasi (Login & Refresh Token), membangun Custom Decorator @Permissions(), dan mengaktifkan PermissionsGuard untuk memotong request ilegal.

Berikut adalah urutan langkah taktis implementasi kodenya:

#### 1. Registrasi Prisma Service ke Modul Global

Agar database dapat dipanggil dengan mudah di semua modul tanpa perlu re-import, pastikan PrismaModule kita mengeksport PrismaService secara global.

Buka `src/infrastructure/prisma/`[prisma.service.ts](#):

```
import { Injectable, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  async onModuleInit() {
    await this.$connect();
  }
}
```

Buka src/infrastructure/prisma/[prisma.module.ts](#):

```
import { Global, Module } from '@nestjs/common';
import { PrismaService } from './prisma.service';
```

```
@Global() // Menjadikan PrismaModule tersedia secara global
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

## 2. Membuat Custom Decorators

Kita memerlukan dekorator penanda untuk memisahkan rute publik dan rute yang membutuhkan izin khusus (granular permissions).

Buka src/common/decorators/[public.decorator.ts](#):

```
import { SetMetadata } from '@nestjs/common';
```

```
export const IS_PUBLIC_KEY = 'isPublic';
export const Public = () => SetMetadata(IS_PUBLIC_KEY, true);
```

Buka src/common/decorators/[permissions.decorator.ts](#):

```
import { SetMetadata } from '@nestjs/common';
```

```
export const PERMISSIONS_KEY = 'permissions';
export const Permissions = (...permissions: string[]) =>
  SetMetadata(PERMISSIONS_KEY, permissions);
```

## 3. Membangun Guard Utama (PermissionsGuard)

Guard ini bertindak sebagai gerbang otorisasi. Ia bertugas menarik data *User* dari *request*, mencuri data *Role* dari database, memetakan *Permission*, lalu mencocokkannya dengan dekorator `@Permissions()`. Jika gagal, sistem otomatis melempar error **403 Forbidden** dan mencatatkan log percobaan ilegal ke **AuditLog**.

Buka [src/common/guards/permissions.guard.ts](#):

```
import { Injectable, CanActivate, ExecutionContext, ForbiddenException } from
 '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { PrismaService } from '../../infrastructure/prisma/prisma.service';
import { PERMISSIONS_KEY } from '../../decorators/permissions.decorator';
```

```
@Injectable()
```

```
export class PermissionsGuard implements CanActivate {
```

```
  constructor(
```

```
    private reflector: Reflector,
```

```
    private prisma: PrismaService,
```

```
  ) {}
```

```
  async canActivate(context: ExecutionContext): Promise<boolean> {
```

```
    // 1. Ambil metadata permission yang diminta oleh controller
```

```
    const requiredPermissions = this.reflector.getAllAndOverride<string[]>(PERMISSIONS_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
```

```
    // Jika endpoint tidak diberi dekorator @Permissions(), izinkan masuk (atau pasang default
    strict)
```

```
    if (!requiredPermissions) {
      return true;
    }
```

```
    // 2. Ambil data user hasil ekstraksi JwtAuthGuard
```

```
    const request = context.switchToHttp().getRequest();
```

```
    const user = request.user;
```

```
    if (!user) {
```

```
      throw new ForbiddenException('Akses ditolak: Identitas pengguna tidak ditemukan.');
```

```
    }
```

```
    // 3. Query ke PostgreSQL untuk menarik semua permission yang dimiliki user berdasarkan
    role-nya
```

```
    const dbUser = await this.prisma.user.findUnique({
```

```
      where: { id: user.sub, deletedAt: null },
```

```
      include: {
```

```
        userRoles: {
```

```
          include: {
```

```
            role: {
```

```

    include: {
      rolePermissions: {
        include: {
          permission: true,
        },
      },
    },
  },
},
},
},
},
});

```

```

if (!dbUser) return false;

```

```

// 4. Flattening struktur data menjadi array string permission biasa
const userPermissions = dbUser.userRoles.flatMap((ur) =>
  ur.role.rolePermissions.map((rp) => rp.permission.name),
);

```

```

// 5. Evaluasi: Apakah user memiliki ALL permission yang diwajibkan?
const hasPermission = requiredPermissions.every((permission) =>
  userPermissions.includes(permission),
);

```

```

if (!hasPermission) {
  // Sesuai Security Checklist: Catat insiden ACCESS_DENIED ke tabel AuditLog
  await this.prisma.auditLog.create({
    data: {
      userId: user.sub,
      action: 'ACCESS_DENIED',
      endpoint: `${request.method} ${request.url}`,
      ipAddress: request.ip,
      userAgent: request.headers['user-agent'],
      status: 'FAILED',
    },
  });
}

```

```

    throw new ForbiddenException('Anda tidak memiliki hak akses yang cukup untuk
    mengeksekusi aksi ini.');
```

```

    return true;
  }
}

```

```
}
```

### Cara Penggunaan di Controller (Contoh Implementasi Modul Permissions)

Kini struktur otorisasi kita sudah berfungsi. Saatnya menggunakannya di dalam controller untuk membatasi fungsionalitas CRUD secara granular sesuai dengan spesifikasi endpoint kita.

Buka **src/modules/permissions/permissions.controller.ts**:

```
import { Controller, Get, Post, Patch, Delete, Body, Param, UseGuards } from '@nestjs/common';
import { Permissions } from '../../common/decorators/permissions.decorator';
import { PermissionsGuard } from '../../common/guards/permissions.guard';
import { JwtAuthGuard } from '../../common/guards/jwt-auth.guard'; // Asumsi guard JWT Anda
```

```
@Controller('permissions')
@UseGuards(JwtAuthGuard, PermissionsGuard) // Mengunci seluruh rute di controller ini
export class PermissionsController {
```

```
  @Get()
  @Permissions('permission:read')
  findAll() {
    return { message: 'Menampilkan semua data permission' };
  }
```

```
  @Post()
  @Permissions('permission:create')
  create(@Body() body: { name: string; resource: string; action: string }) {
    return { message: 'Permission baru berhasil dibuat', data: body };
  }
```

```
  @Patch(':id')
  @Permissions('permission:update')
  update(@Param('id') id: string, @Body() body: any) {
    return { message: `Permission ID ${id} berhasil diperbarui` };
  }
```

```
  @Delete(':id')
  @Permissions('permission:delete')
  remove(@Param('id') id: string) {
    return { message: `Permission ID ${id} berhasil dihapus` };
  }
}
```

## Sprint 3: Otorisasi & Logika Bisnis 2 (Core Implementation AuthService dan AuthController)

### Goal Tahapan Ini

- Menangani validasi password menggunakan bcrypt.
- Menghasilkan sepasang token: Access Token (pendek) dan Refresh Token (panjang).
- Menyimpan versi hash dari Refresh Token ke PostgreSQL via Prisma.

### Timeline & Goal Sprint 3

- Durasi: 3–4 Hari Kerja
- Target Utama: Menyelesaikan modul autentikasi (Login & Refresh Token), membangun Custom Decorator @Permissions(), dan mengaktifkan PermissionsGuard untuk memotong request ilegal.

Panduan eksekusi taktis untuk melengkapi AuthService dan menyusun sistem otentikasi hibrida (Access Token + Hashed Refresh Token) sesuai dengan standar industri dan instruksi Security Checklist.

#### 1. Implementasi AuthService

Buka dan perbarui berkas src/modules/auth/auth.service.ts kita dengan logika otentikasi berikut:

```
import { Injectable, UnauthorizedException, BadRequestException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { ConfigService } from '@nestjs/config';
import { PrismaService } from '../../infrastructure/prisma/prisma.service';
import * as bcrypt from 'bcrypt';
```

```
@Injectable()
```

```
export class AuthService {
```

```
  constructor(
```

```
    private prisma: PrismaService,
```

```
    private jwtService: JwtService,
```

```
    private configService: ConfigService,
```

```
  ) {}
```

```
  /**
```

```
   * Fungsi Login Utama
```

```
   */
```

```
  async login(email: string, passwordPlain: string, ip: string, ua: string) {
```

```
    // 1. Cari user berdasarkan email
```

```
    const user = await this.prisma.user.findUnique({
      where: { email, deletedAt: null },
    });
```

```
    // 2. Sesuai Security Checklist: Verifikasi password menggunakan bcrypt
```

```
    if (!user || !(await bcrypt.compare(passwordPlain, user.password))) {
```

```
      // Catat insiden kegagalan login ke Audit Log
```

```

await this.prisma.auditLog.create({
  data: {
    action: 'AUTH_LOGIN_FAILED',
    endpoint: 'POST /auth/login',
    ipAddress: ip,
    userAgent: ua,
    status: 'FAILED',
  },
});
throw new UnauthorizedException('Email atau password yang Anda masukkan salah.');
```

```

// 3. Generate pasangan token (Access & Refresh)
const tokens = await this.generateTokens(user.id, user.email);
```

```

// 4. Sesuai Security Checklist: Hash Refresh Token sebelum masuk ke database
const saltRounds = 10;
const hashedRefreshToken = await bcrypt.hash(tokens.refreshToken, saltRounds);
```

```

// 5. Simpan ke tabel refresh_tokens
const refreshExpiresInDays = 30; // Selaras dengan JWT_REFRESH_EXPIRES=30d
await this.prisma.refreshToken.create({
  data: {
    userId: user.id,
    tokenHash: hashedRefreshToken,
    expiresAt: new Date(Date.now() + refreshExpiresInDays * 24 * 60 * 60 * 1000),
  },
});
```

```

// 6. Catat log keberhasilan login ke Audit Log
await this.prisma.auditLog.create({
  data: {
    userId: user.id,
    action: 'AUTH_LOGIN_SUCCESS',
    endpoint: 'POST /auth/login',
    ipAddress: ip,
    userAgent: ua,
    status: 'SUCCESS',
  },
});
```

```

return {
  userId: user.id,
  email: user.email,
```

```

    ...tokens,
  };
}

/**
 * Fungsi Regenerasi Access Token via Refresh Token
 */
async refreshTokens(userId: string, refreshTokenPlain: string, ip: string, ua: string) {
  // 1. Ambil semua token aktif milik user dari DB
  const activeTokens = await this.prisma.refreshToken.findMany({
    where: { userId, expiresAt: { gte: new Date() } },
  });

  // 2. Cari token mana yang cocok dengan plaintext token dari client
  let validTokenRecord = null;
  for (const record of activeTokens) {
    const isMatch = await bcrypt.compare(refreshTokenPlain, record.tokenHash);
    if (isMatch) {
      validTokenRecord = record;
      break;
    }
  }

  if (!validTokenRecord) {
    throw new UnauthorizedException('Refresh token tidak valid atau sudah kedaluwarsa.');
```

```

    return tokens;
  }

  /**
   * Helper internal untuk menerbitkan token JWT
   */
  private async generateTokens(userId: string, email: string) {
    const jwtPayload = { sub: userId, email };

    const [accessToken, refreshToken] = await Promise.all([
      this.jwtService.signAsync(jwtPayload, {
        secret: this.configService.get<string>('JWT_ACCESS_SECRET') || 'defaultAccess',
        expiresIn: this.configService.get<string>('JWT_ACCESS_EXPIRES') || '15m',
      }),
      this.jwtService.signAsync(jwtPayload, {
        secret: this.configService.get<string>('JWT_REFRESH_SECRET') || 'defaultRefresh',
        expiresIn: this.configService.get<string>('JWT_REFRESH_EXPIRES') || '30d',
      }),
    ]);

    return { accessToken, refreshToken };
  }
}

```

## 2. Membuat Endpoint di AuthController

Sekarang buat atau perbarui berkas `src/modules/auth/auth.controller.ts` untuk mengekspos logika di atas menjadi rute REST API yang bisa diakses oleh frontend maupun diuji via Postman.

```

import { Controller, Post, Body, Req, UnauthorizedException } from '@nestjs/common';
import { AuthService } from './auth.service';
import { Request } from 'express';
import { Public } from '../../common/decorators/public.decorator';

```

```

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

```

```

  @Public() // Dekorator penanda bahwa endpoint ini bypass JwtAuthGuard global

```

```

  @Post('login')
  async login(
    @Req() req: Request,
    @Body() body: { email: string; passwordPlain: string },
  ) {
    return this.authService.login(
      body.email,

```

```

        body.passwordPlain,
        req.ip,
        req.headers['user-agent'],
    );
}

@Public()
@Post('refresh')
async refresh(
    @Req() req: Request,
    @Body() body: { userId: string; refreshToken: string },
) {
    if (!body.userId || !body.refreshToken) {
        throw new UnauthorizedException('Parameter userId dan refreshToken wajib disertakan.');
```

Sprint 3: Otorisasi & Logika Bisnis 1 dan 2 dibuat :

Jika pengaman (PermissionsGuard dan decorator) yang kita buat di awal Sprint 3 bertindak sebagai "Satpam/Sistem Gerbang", maka kode AuthService dan AuthController ini dibuat untuk bertindak sebagai "Loket Pembuatan KTP & Kartu Akses".

Secara rinci, bagian ini mengimplementasikan 3 pilar penting dalam Security Checklist:

### 1. Autentikasi Pengguna Aman (**POST /auth/login**)

- **Pencocokan Kredensial:** Memastikan email terdaftar dan membandingkan password teks biasa dari klien dengan password terenkripsi di PostgreSQL menggunakan **bcrypt.compare**.
- **Penerbitan Akses Berlapis:** Jika cocok, sistem menerbitkan sepasang token:
  - **Access Token:** Kartu akses jangka pendek (15 menit) untuk dibawa klien saat menembak API.
  - **Refresh Token:** Token jangka panjang (30 hari) untuk meminta kartu akses baru jika yang lama kedaluwarsa.

### 2. Kepatuhan Enkripsi Token Relasional (**Refresh Token Hash**)

- Sesuai standar industri, **Refresh Token tidak boleh disimpan dalam bentuk teks biasa di database.**

- Kode di atas otomatis melakukan *hashing* pada Refresh Token sebelum disimpan ke tabel `refresh_tokens`. Jadi, jika database diserang atau bocor, peretas tidak bisa menggunakan data curian tersebut untuk masuk ke sistem.

### 3. Siklus Rotasi Token Tunggal (`POST /auth/refresh`)

- Melindungi dari pembajakan sesi. Saat klien menukarkan Refresh Token lama untuk meminta Access Token baru, **Refresh Token lama tersebut langsung dihapus secara permanen** dari database, lalu sistem menerbitkan Refresh Token baru yang telah di-hash ulang.

### 4. Pencatatan Audit Log Otomatis

- Setiap kali ada upaya login yang berhasil (`AUTH_LOGIN_SUCCESS`) maupun yang gagal (`AUTH_LOGIN_FAILED`), sistem langsung merekam IP Address, waktu, dan jenis perangkat (*User-Agent*) penyerang/pengguna ke tabel `audit_logs` untuk forensik keamanan.

## Sprint 4: Database Seeding & Initial Access Verification.

Sprint ini adalah jembatan krusial untuk mengisi data awal (master data) sehingga sistem otorisasi RBAC dan akun administrator utama (Super Admin) dapat digunakan untuk pertama kali baik di lingkungan lokal maupun siap saat dideploy ke produksi.

Timeline & Goal Sprint 4

- Durasi: 1–2 Hari Kerja
- Sprint Goal: Menyediakan mekanisme pengisian data otomatis (automated seeding) untuk melahirkan akun `SUPER_ADMIN` pertama yang memiliki hak akses penuh, serta memverifikasi bahwa siklus login dan pemblokiran `PermissionsGuard` berjalan 100% valid saat diuji.

### Panduan Eksekusi Langkah Demi Langkah

#### 1. Membuat Skrip Seed (`database/seed/main.ts`)

Karena kita menaruh folder `database/` di luar folder `src/` sesuai dengan struktur folder final Anda, kita akan membuat skrip TypeScript mandiri yang memanfaatkan *Prisma Client* untuk menyuntikkan data secara berurutan (*sequential transactional insert*).

Buat file baru di `database/seed/main.ts`:

```
import { PrismaClient } from '@prisma/client';
import * as bcrypt from 'bcrypt';
```

```
const prisma = new PrismaClient();
```

```
async function main() {
```

```
console.log('🔄 Memulai proses seeding data master keamanan...');
```

```
// 1. Buat Master Permissions
```

```
const permissionsData = [  
  { name: 'user:create', resource: 'user', action: 'create' },  
  { name: 'user:read', resource: 'user', action: 'read' },  
  { name: 'user:update', resource: 'user', action: 'update' },  
  { name: 'user:delete', resource: 'user', action: 'delete' },  
  { name: 'permission:read', resource: 'permission', action: 'read' },  
];
```

```
const permissions = [];  
for (const p of permissionsData) {  
  const permission = await prisma.permission.upsert({  
    where: { name: p.name },  
    update: {},  
    create: p,  
  });  
  permissions.push(permission);  
  console.log(` ✓ Permission dibuat: ${p.name}`);  
}
```

```
// 2. Buat Master Role: SUPER_ADMIN
```

```
const superAdminRole = await prisma.role.upsert({  
  where: { name: 'SUPER_ADMIN' },  
  update: {},  
  create: {  
    name: 'SUPER_ADMIN',  
    description: 'Pemilik sistem tertinggi dengan akses seluruh modul',  
  },  
});  
console.log(` ✓ Role dibuat: SUPER_ADMIN`);
```

```
// 3. Hubungkan Role dengan ALL Permissions (RolePermission Junction)
```

```
for (const perm of permissions) {  
  await prisma.rolePermission.upsert({  
    where: {  
      roleId_permissionId: {  
        roleId: superAdminRole.id,  
        permissionId: perm.id,  
      },  
    },  
    update: {},  
    create: {
```

```

        roleId: superAdminRole.id,
        permissionId: perm.id,
      },
    });
  }
  console.log(' ✓ Relasi Role-Permission berhasil diikat.');
```

// 4. Sesuai Security Checklist: Hash password admin (12 rounds)

```
const hashedPassword = await bcrypt.hash('SuperAdminSecure2026!', 12);
```

// 5. Buat Akun User Super Admin Utama

```
const adminUser = await prisma.user.upsert({
  where: { email: 'admin@domain.com' },
  update: {},
  create: {
    email: 'admin@domain.com',
    password: hashedPassword,
    name: 'Utama Super Admin',
  },
});
console.log(` ✓ User Admin dibuat: ${adminUser.email}`);
```

// 6. Hubungkan User dengan Role SUPER\_ADMIN (UserRole Junction)

```
await prisma.userRole.upsert({
  where: {
    userId_roleId: {
      userId: adminUser.id,
      roleId: superAdminRole.id,
    },
  },
  update: {},
  create: {
    userId: adminUser.id,
    roleId: superAdminRole.id,
  },
});
console.log(' ✓ Akses SUPER_ADMIN resmi diberikan kepada user.');
```

```
console.log('✅ Proses seeding selesai dengan sukses!');
```

```
}

main()
  .catch((e) => {
    console.error('❌ Terjadi kesalahan saat seeding:', e);
  });
```

```
process.exit(1);
})
.finally(async () => {
  await prisma.$disconnect();
});
```

## 2. Mendaftarkan Skrip Seed ke **package.json**

Agar skrip di atas dapat dipanggil secara otomatis oleh CLI Prisma, tambahkan konfigurasi pointer **ts-node** ke dalam file **package.json** Anda pada tingkat root objek:

```
"prisma": {
  "seed": "ts-node database/seed/main.ts"
}
```

## 3. Mengeksekusi Seeding ke PostgreSQL

Pastikan container database Anda menyala, lalu jalankan perintah berikut di terminal:

```
npx prisma db seed
```

Output di terminal akan memunculkan tanda centang hijau (✓) dari skrip yang menandakan data berhasil masuk.

## Pengujian & Verifikasi Akses (Aksi Nyata)

Untuk memastikan seluruh rangkaian dari Sprint 1 hingga Sprint 4 berjalan tanpa cacat, lakukan uji coba integrasi berikut menggunakan Postman/Insomnia:

Pengujian A: Proses Login Utama

Tembak rute **POST** <http://localhost:3000/auth/login>.

Kirim JSON Body berikut:

```
{
  "email": "admin@domain.com",
  "passwordPlain": "SuperAdminSecure2026!"
}
```

**Ekspektasi Hasil:** kita mendapatkan respons **200 OK** yang berisi string **accessToken** dan **refreshToken**. Salin string **accessToken** tersebut.

### 1. Pengujian B: Akses Endpoint dengan Hak Akses (Lolos Guard)

2. Tembak rute **GET** <http://localhost:3000/permissions> (Endpoint yang kita buat di Sprint 3).

3. Masukkan token Anda di tab **Authorization** sebagai **Bearer Token**.

4. **Ekspektasi Hasil:** Respons **200 OK** dengan pesan "**Menampilkan semua data permission**". Mengapa lolos? Karena user **admin@domain.com** terikat pada role **SUPER\_ADMIN** yang memiliki permission **permission:read**.

#### **Pengujian C: Pengecekan Audit Log Keamanan**

1. Buka kembali **Prisma Studio** Anda (**npx prisma studio**).
2. Masuk ke model/tabel **AuditLog**.
3. **Ekspektasi Hasil:** Anda akan melihat record log bertuliskan **AUTH\_LOGIN\_SUCCESS** lengkap dengan data IP Address (**::1** atau **127.0.0.1**) dan User-Agent perangkat kita.

### **S.SPRINT IAM RBAC SERVICE ZAFI PROPERTI**

Targetnya **1 minggu selesai menjadi RBAC API Service Production Ready**, menggunakan **Daily Sprint Plan (7 Hari)** dengan deliverable yang jelas setiap hari.

## **Roadmap 7 Hari (Production Ready)**

### **Hari 1 — Foundation & Project Setup**

#### **Goal**

Membangun pondasi project yang stabil dan siap dikembangkan.

#### **Output**

- ✓ NestJS Setup
- ✓ Prisma Setup
- ✓ PostgreSQL Setup
- ✓ Docker Setup
- ✓ Config Module
- ✓ Swagger Setup
- ✓ Logger Setup
- ✓ GitHub Repository
- ✓ First Commit

#### **Coding Tasks**

#### **Struktur Folder**

Buat seluruh struktur folder final terlebih dahulu.

```
src/  
config/  
common/  
infrastructure/  
modules/  
shared/  
database/
```

### Install Dependency

```
npm install @nestjs/config  
npm install @nestjs/swagger  
npm install swagger-ui-express
```

```
npm install @prisma/client  
npm install prisma@5
```

```
npm install winston  
npm install nest-winston
```

```
npm install class-validator  
npm install class-transformer
```

```
npm install helmet  
npm install @nestjs/throttler
```

### Prisma

Buat:

```
prisma/  
├── schema.prisma  
├── migrations/  
└── seed.ts
```

### Commit

```
git commit -m "chore: initialize project structure"
```

## Hari 2 — Database Design & Prisma Layer

### Goal

Database final dan siap migrasi.

### Output

- ✓ User
- ✓ Role
- ✓ Permission
- ✓ UserRole
- ✓ RolePermission
- ✓ RefreshToken
- ✓ AuditLog

### **Coding Tasks Prisma Models , Buat model:**

1. User
2. Role
3. Permission
4. UserRole
5. RolePermission
6. RefreshToken
7. AuditLog

### **Migration**

`npx prisma migrate dev`

### **Seeder**

Role:

1. SUPER\_ADMIN
2. ADMIN
3. STAFF

Permission:

user:create  
user:view  
user:update  
user:delete

### **Commit**

`git commit -m "feat: add database schema and seeders"`

## **Hari 3 — User Module**

### **Goal**

CRUD User selesai.

### **Output**

- ✓ Create User
- ✓ List User
- ✓ Detail User
- ✓ Update User
- ✓ Soft Delete User

### **Struktur**

modules/users

controllers/  
services/  
repositories/  
dto/  
entities/

### **Coding Tasks**

Buat:

CreateUserDto  
UpdateUserDto

UsersController  
UsersService  
UsersRepository

### **Checklist**

Validation Pipe  
Pagination  
Search  
Soft Delete

### **Commit**

git commit -m "feat: complete user module"

## **Hari 4 — Auth Module**

### **Goal**

Authentication selesai.

### **Output**

- ✓ Register
- ✓ Login
- ✓ Profile
- ✓ Logout

✓ Refresh Token

## Coding Tasks

### Install

```
npm install @nestjs/jwt  
npm install passport  
npm install passport-jwt  
npm install bcrypt
```

### Buat

JwtStrategy

AuthService

AuthController

### Endpoint

```
POST /auth/register  
POST /auth/login  
POST /auth/refresh  
POST /auth/logout  
GET /auth/profile
```

### Checklist

Password Hash  
JWT Access Token  
Refresh Token Hash

### Commit

```
git commit -m "feat: implement authentication"
```

## Hari 5 — RBAC Module

### Goal

Role dan Permission selesai.

### Output

✓ Roles CRUD

- ✓ Permissions CRUD
- ✓ Assign Role
- ✓ Assign Permission
- ✓ Guards
- ✓ Decorators

## **Coding Tasks**

### **Modules**

roles  
permissions  
user-roles  
role-permissions

### **Common**

roles.decorator.ts  
permissions.decorator.ts

roles.guard.ts  
permissions.guard.ts

### **Endpoint**

POST /user-roles

POST /role-permissions

### **Checklist**

@Roles()

@Permissions()

### **Commit**

git commit -m "feat: implement RBAC authorization"

## **Hari 6 — Audit Log & Security**

### **Goal**

Hardening aplikasi.

### **Output**

- ✓ Audit Log
- ✓ Helmet
- ✓ Throttler
- ✓ Global Exception
- ✓ Response Formatter

## **Coding Tasks**

### **Common**

http-exception.filter.ts

transform.interceptor.ts

### **Audit**

audit-logs module

### **Security**

app.use(helmet());

ThrottlerModule.forRoot()

## **Checklist**

Rate Limit

Validation

Audit Logging

Consistent Response

## **Commit**

git commit -m "feat: add audit logs and security layer"

## **Hari 7 — Production & DevOps**

### **Goal**

Deployable ke VPS.

### **Output**

- ✓ Docker

- ✓ Docker Compose
- ✓ GitHub Actions
- ✓ Nginx Config
- ✓ README
- ✓ Postman Collection

## **Docker**

postgres  
api  
redis

## **GitHub Actions**

Pipeline

Lint

↓

Test

↓

Build

↓

Docker Build

↓

Push GHCR

↓

Deploy VPS

## **VPS**

Ubuntu 24.04  
Docker  
Docker Compose  
Nginx  
SSL

## **Commit**

git commit -m "chore: production deployment setup"

# Definition of Done (DoD)

Pada akhir minggu, aplikasi harus memenuhi checklist berikut:

| Area           | Status |
|----------------|--------|
| NestJS Setup   | ✓      |
| Prisma 5       | ✓      |
| PostgreSQL     | ✓      |
| Docker         | ✓      |
| Swagger        | ✓      |
| User CRUD      | ✓      |
| Auth JWT       | ✓      |
| Refresh Token  | ✓      |
| Roles          | ✓      |
| Permissions    | ✓      |
| RBAC Guard     | ✓      |
| Audit Log      | ✓      |
| Rate Limit     | ✓      |
| GitHub Actions | ✓      |
| VPS Deployment | ✓      |

## Panduan Clean Architecture untuk Setiap Modul

Setiap module wajib mengikuti pola yang sama:

```
modules/  
├── users/  
│   ├── controllers/  
│   │   └── users.controller.ts  
│   ├── services/  
│   │   └── users.service.ts  
│   ├── repositories/  
│   │   └── users.repository.ts  
│   ├── dto/  
│   │   ├── create-user.dto.ts  
│   │   └── update-user.dto.ts  
│   └── entities/  
│       └── user.entity.ts
```

└─ users.module.ts

**Rule dependency:**

Controller  
↓  
Service  
↓  
Repository  
↓  
PrismaService  
↓  
PostgreSQL

**Tidak boleh:**

Controller  
↓  
PrismaService

**atau**

Controller  
↓  
Repository

**PANDUAN TDC:**

<https://drive.google.com/file/d/19hydAifzkG0MnCG3coMRG5FDZmZvZ0RW/view?usp=sharing>

- I.
- J.
- K.
- L.
- M.